



Первое высшее техническое учебное заведение России Санкт-Петербургский горный университет императрицы Екатерины II



Основы конструирования измерительных систем

Лекция 1. Общие сведения о микроконтроллерах и цифровом сигнале

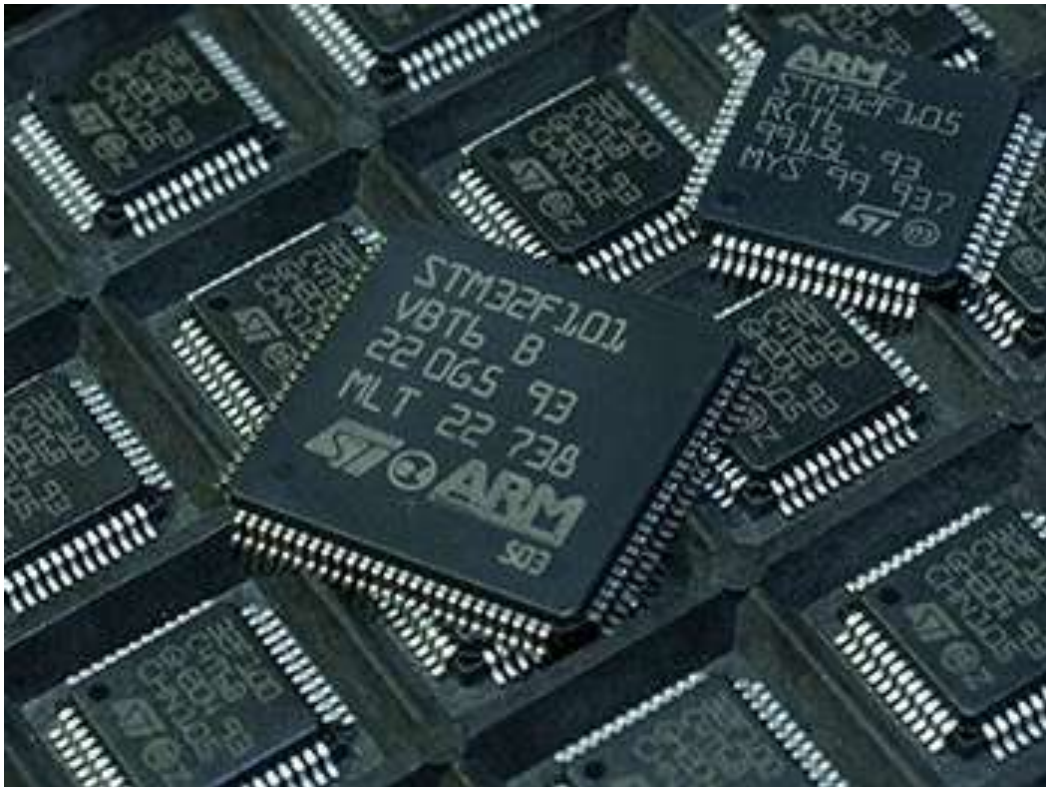
М.Г. Выстрчил, к.т.н., доцент каф. Маркшейдерского дела

г. Санкт-Петербург
2026

Микроконтроллеры и микрокомпьютеры

Микроконтро́ллер (англ. Micro Controller Unit, MCU) — микросхема для программного управления электронными устройствами.

Обычно изготавливается в виде единого кристалла с функциями ядра микропроцессора, шин команд и данных, периферийных устройств, ОЗУ и ПЗУ. По сути, это компьютер в чипе, выполняющий роль периферийного процессора.

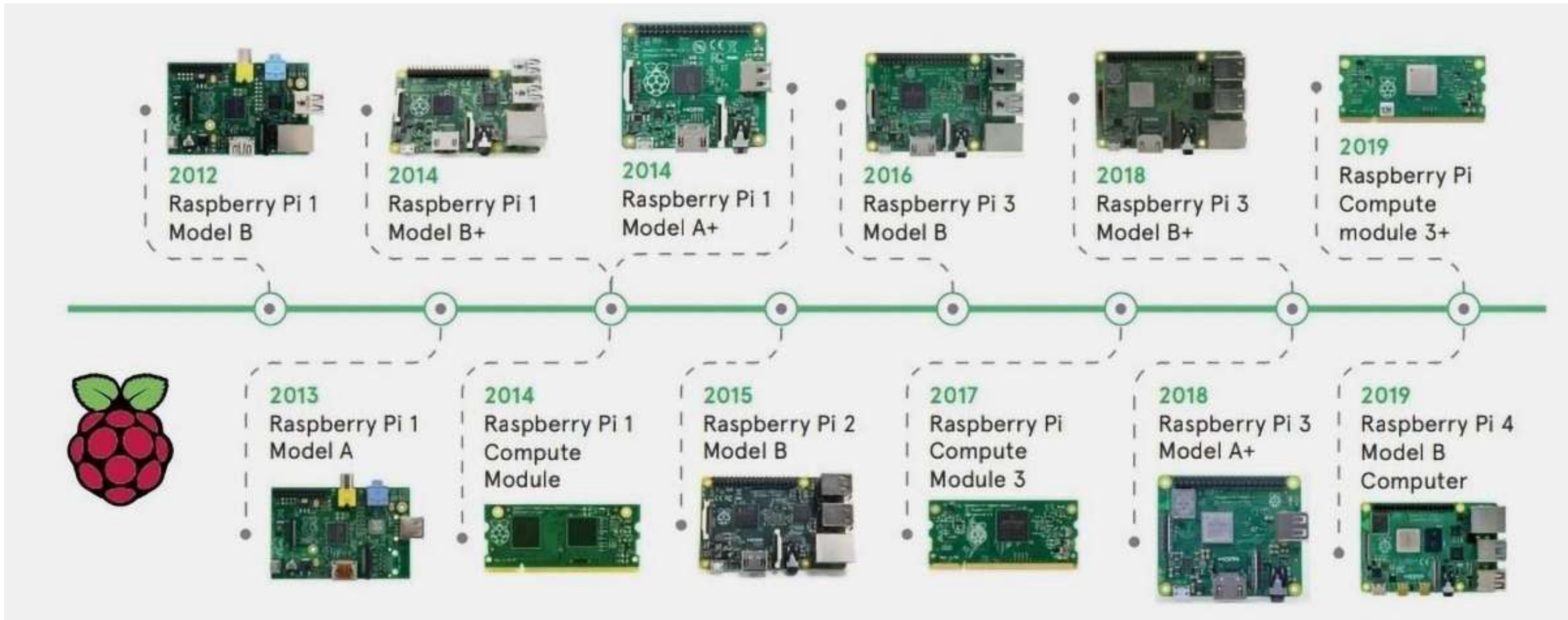


Микроконтроллеры и микрокомпьютеры

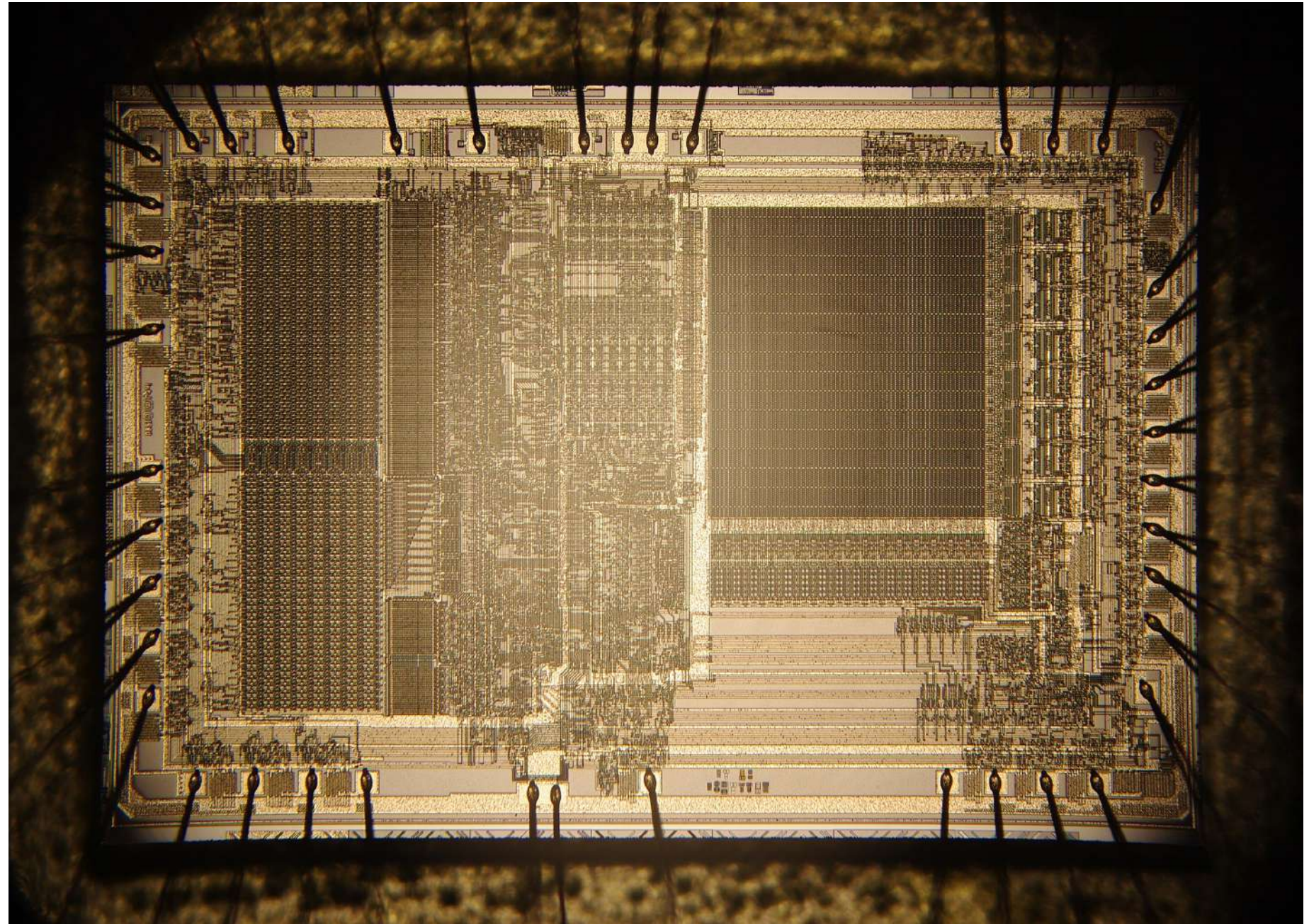
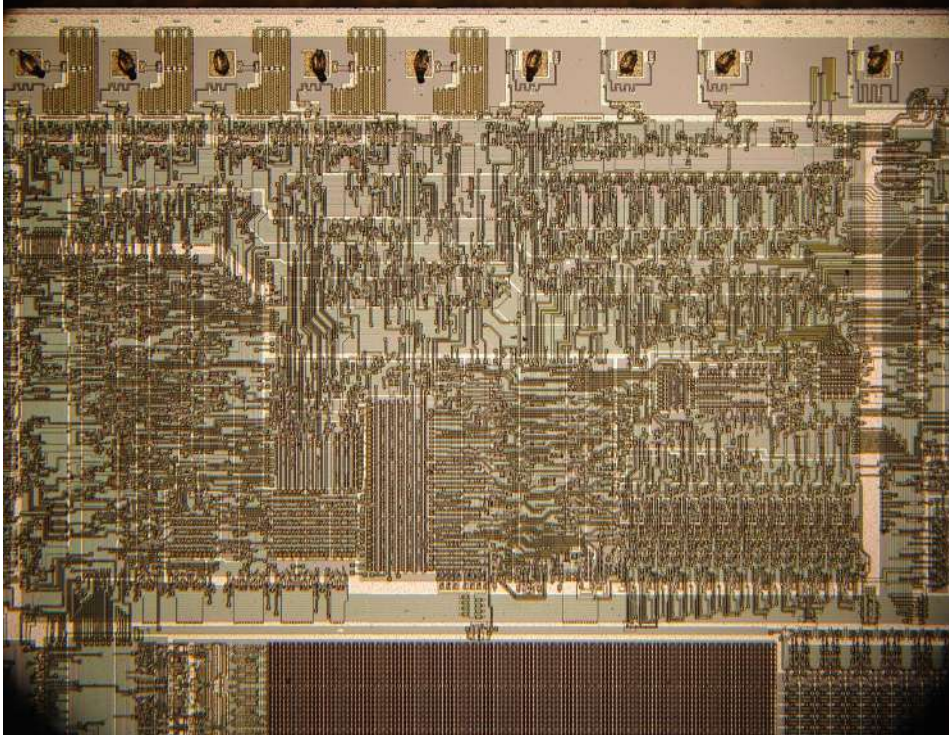
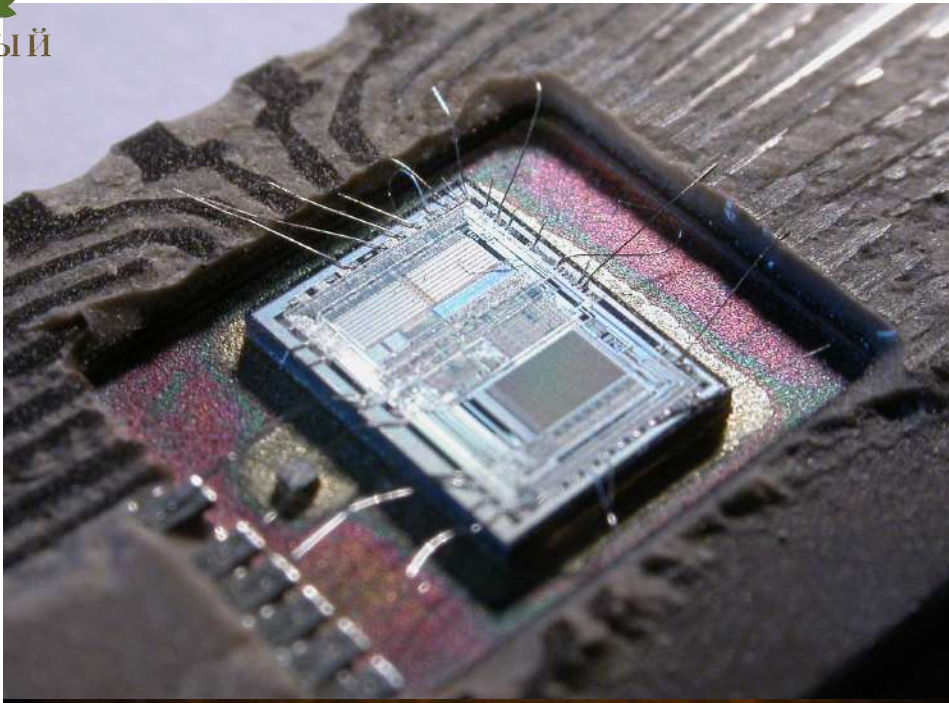
Одноплатный компьютер (SBC, англ. single-board computer) — самодостаточный компьютер, собранный на одной печатной плате, на которой установлены микропроцессор, оперативная память, системы ввода-вывода и другие модули, необходимые для функционирования компьютера.



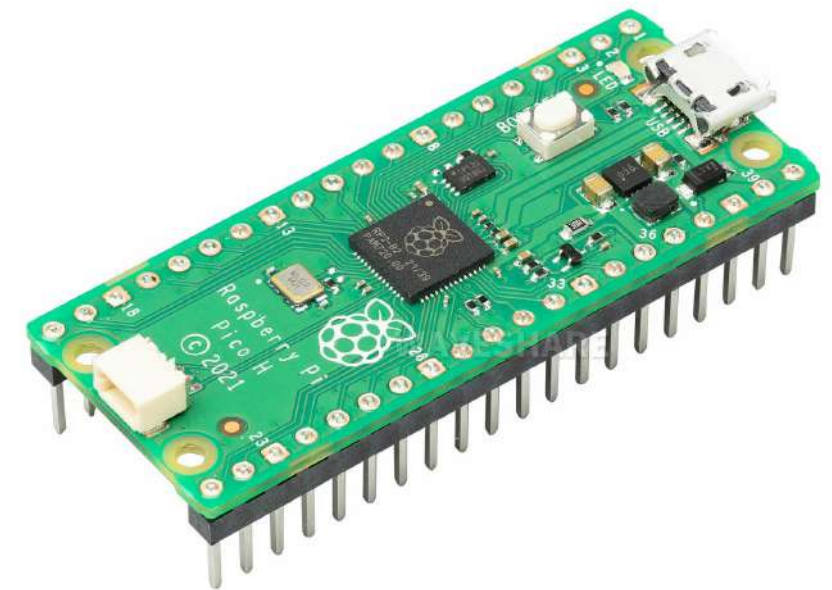
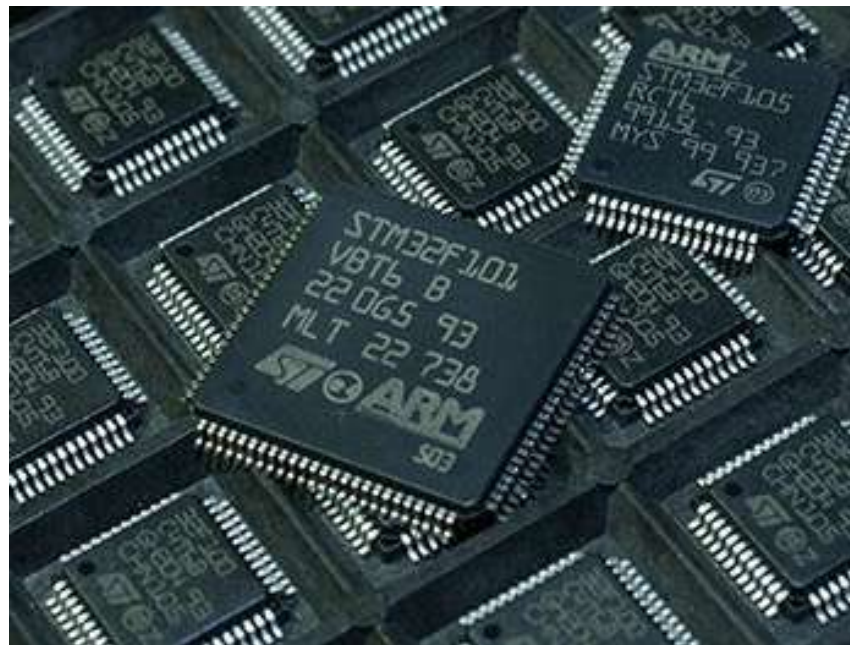
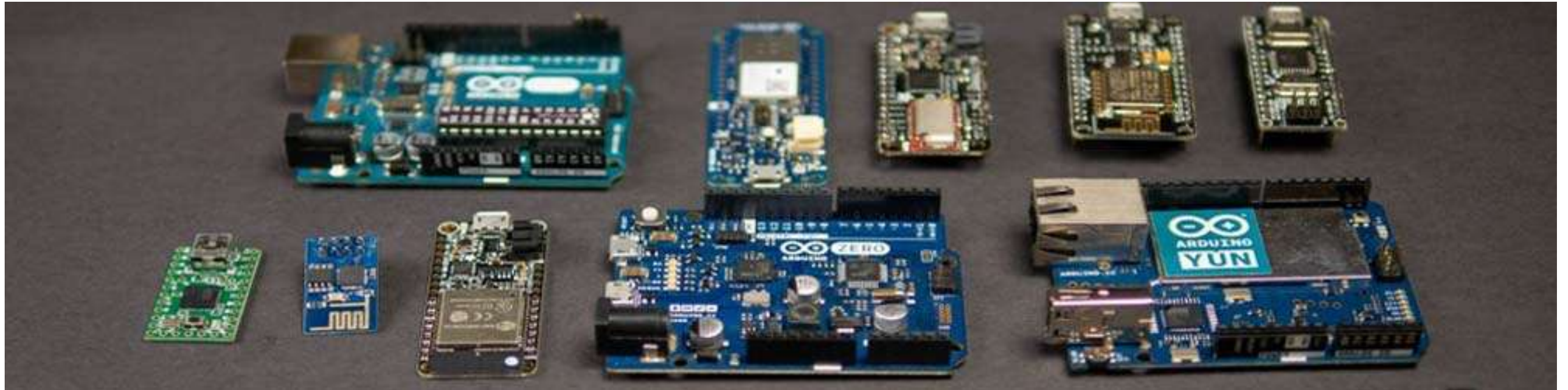
Микроконтроллеры и микрокомпьютеры



Микроконтроллеры и микрокомпьютеры



Микроконтроллеры и микрокомпьютеры



Микроконтроллеры и микрокомпьютеры





Микроконтроллеры и микрокомпьютеры



RP2040



A microcontroller chip designed by
Raspberry Pi

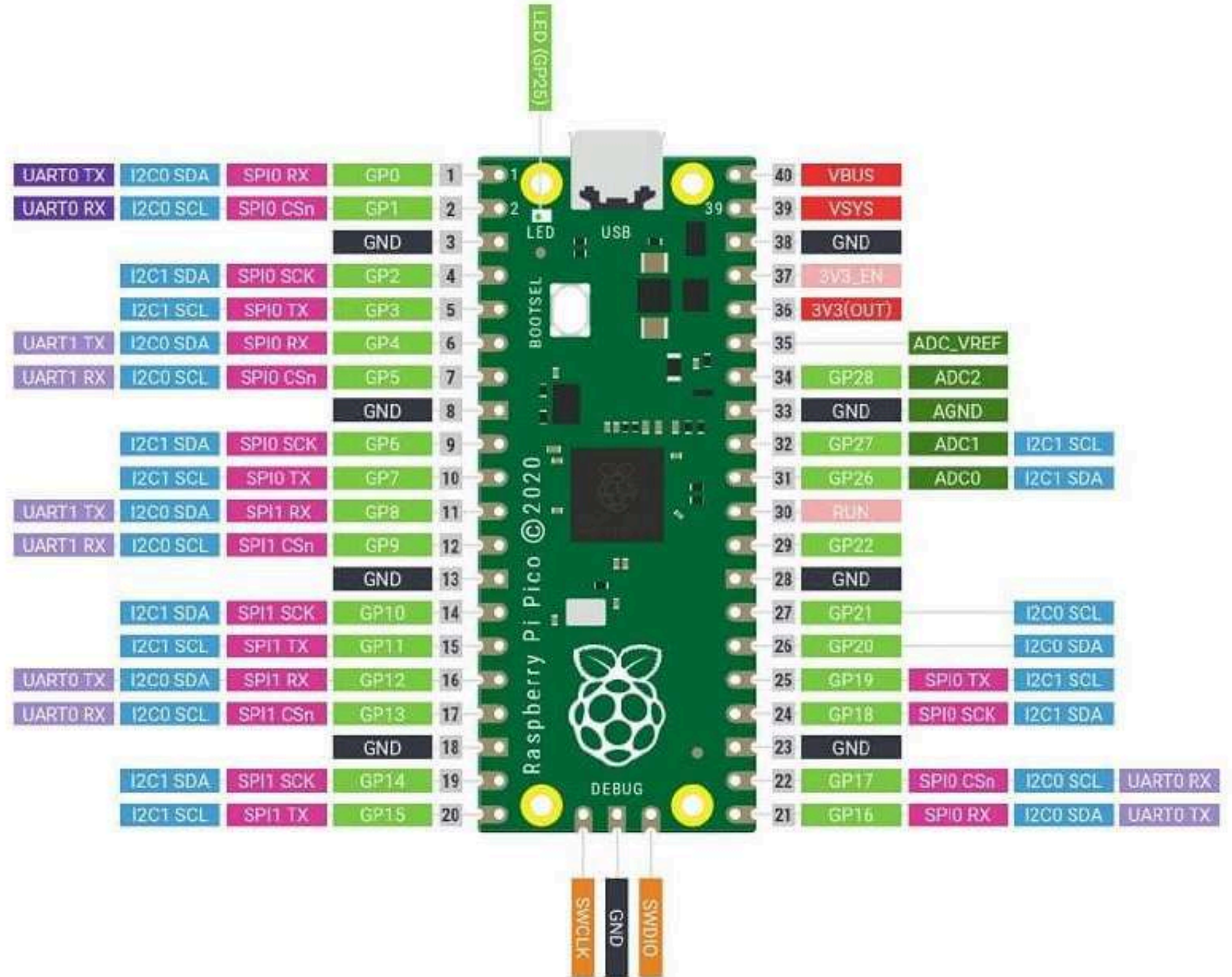


Микроконтроллеры и микрокомпьютеры

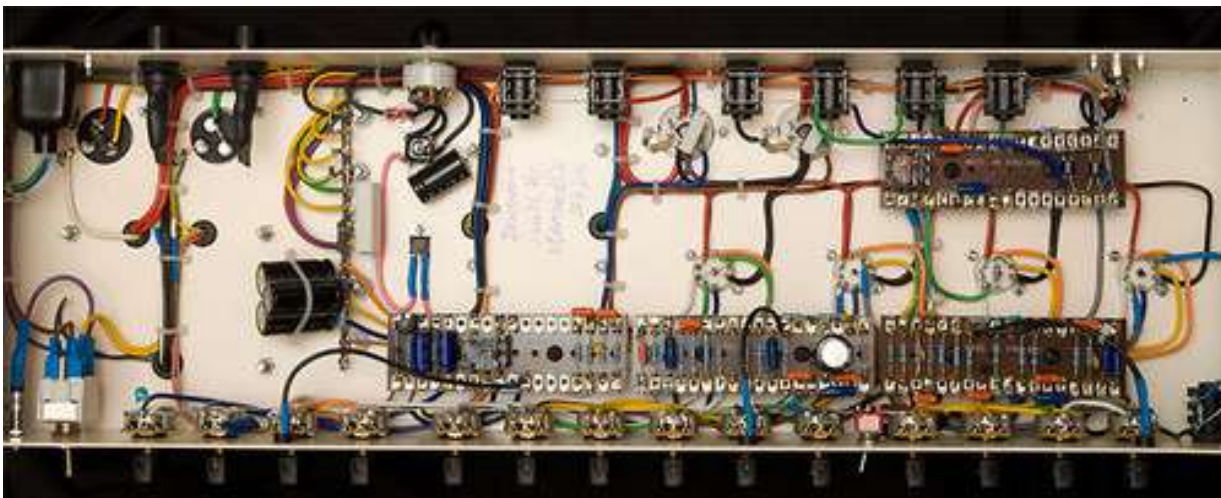
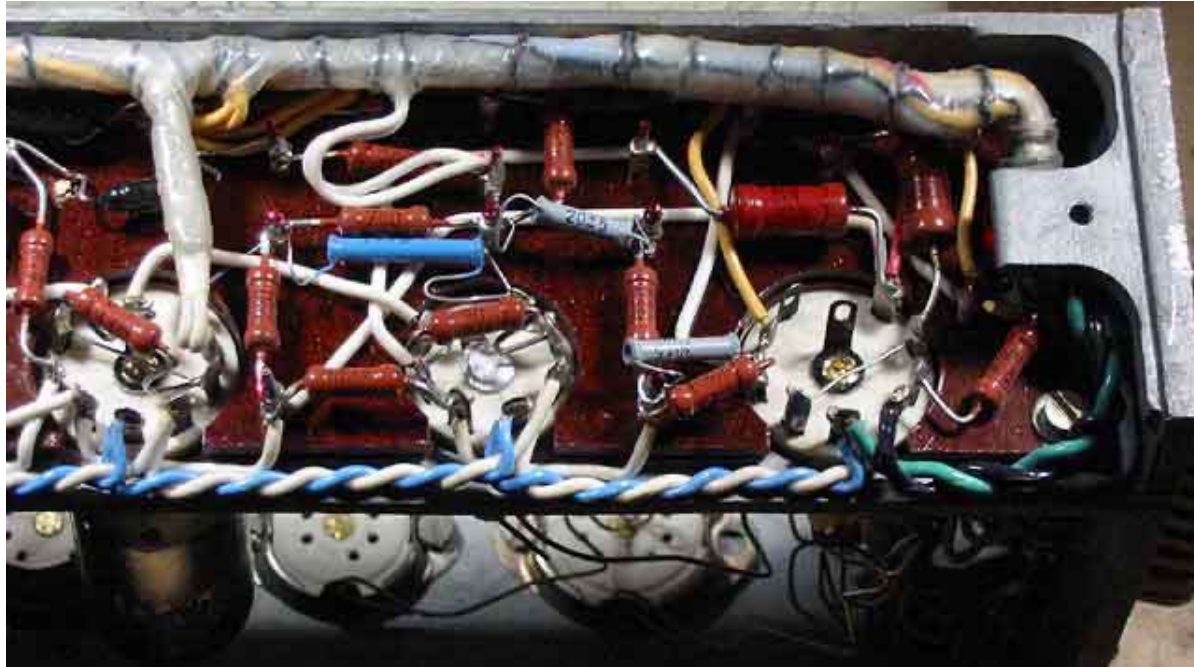
Микроконтроллер: RP2040

- Процессор: ARM Cortex-M0+ с частотой до 133 МГц
- Память:
 - 264 КБ SRAM
 - Нет встроенной памяти EEPROM или Flash, но есть возможность подключения внешней памяти через SPI или I2C
- GPIO (общее число): 26
- Аналоговые входы: 3 (дисковые резисторы)
- UART, SPI, I2C, PWM доступны на разъемах GPIO
- USB 1.1 с поддержкой битрейта до 480 Мбит/сек
- Поддержка работы с внешними устройствами через PIO (Programmable Input/Output)
- Поддержка множества языков программирования, включая C, MicroPython и др.
- Напряжение питания: 3,3 В
- Размеры: 51 x 21 мм

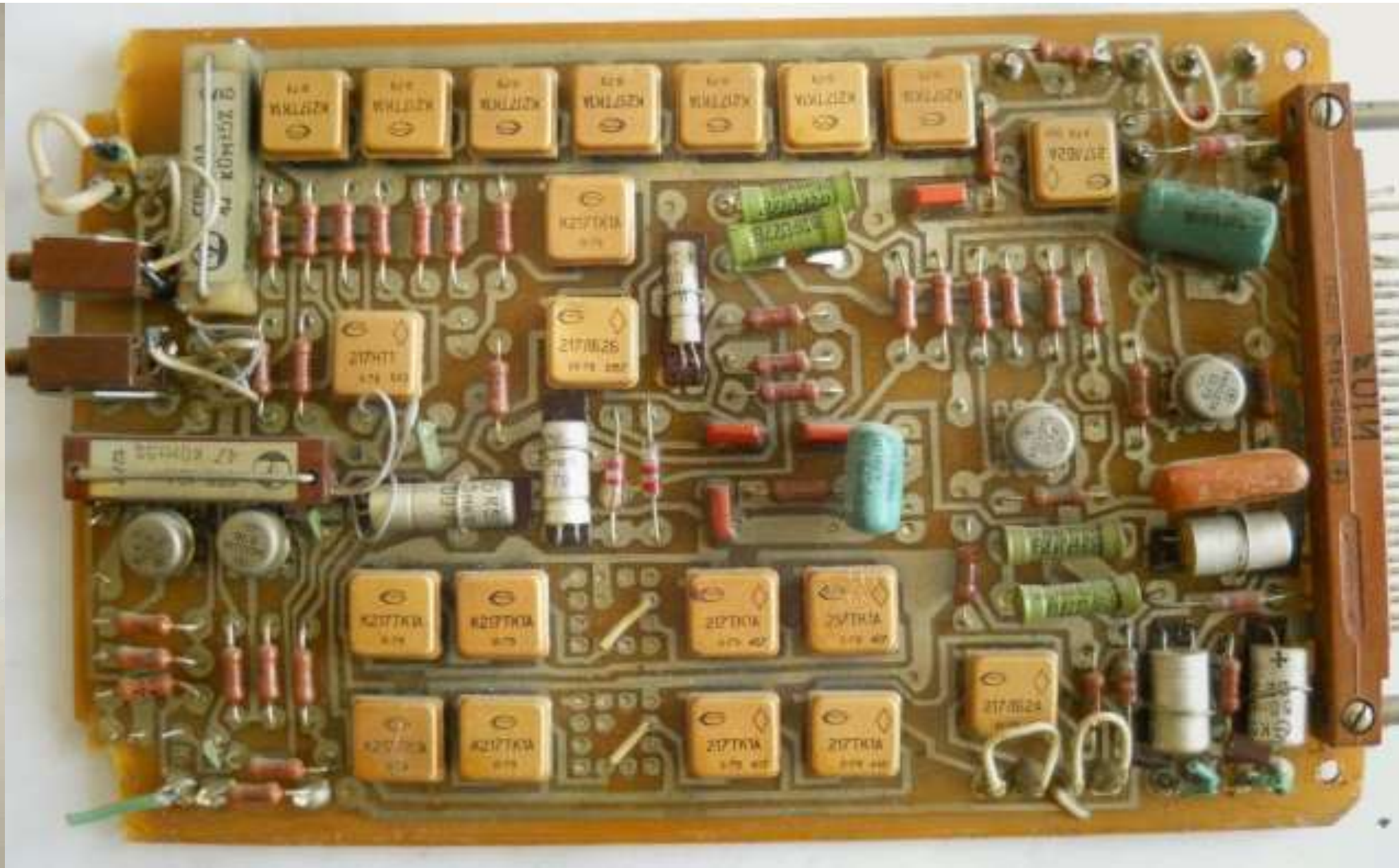
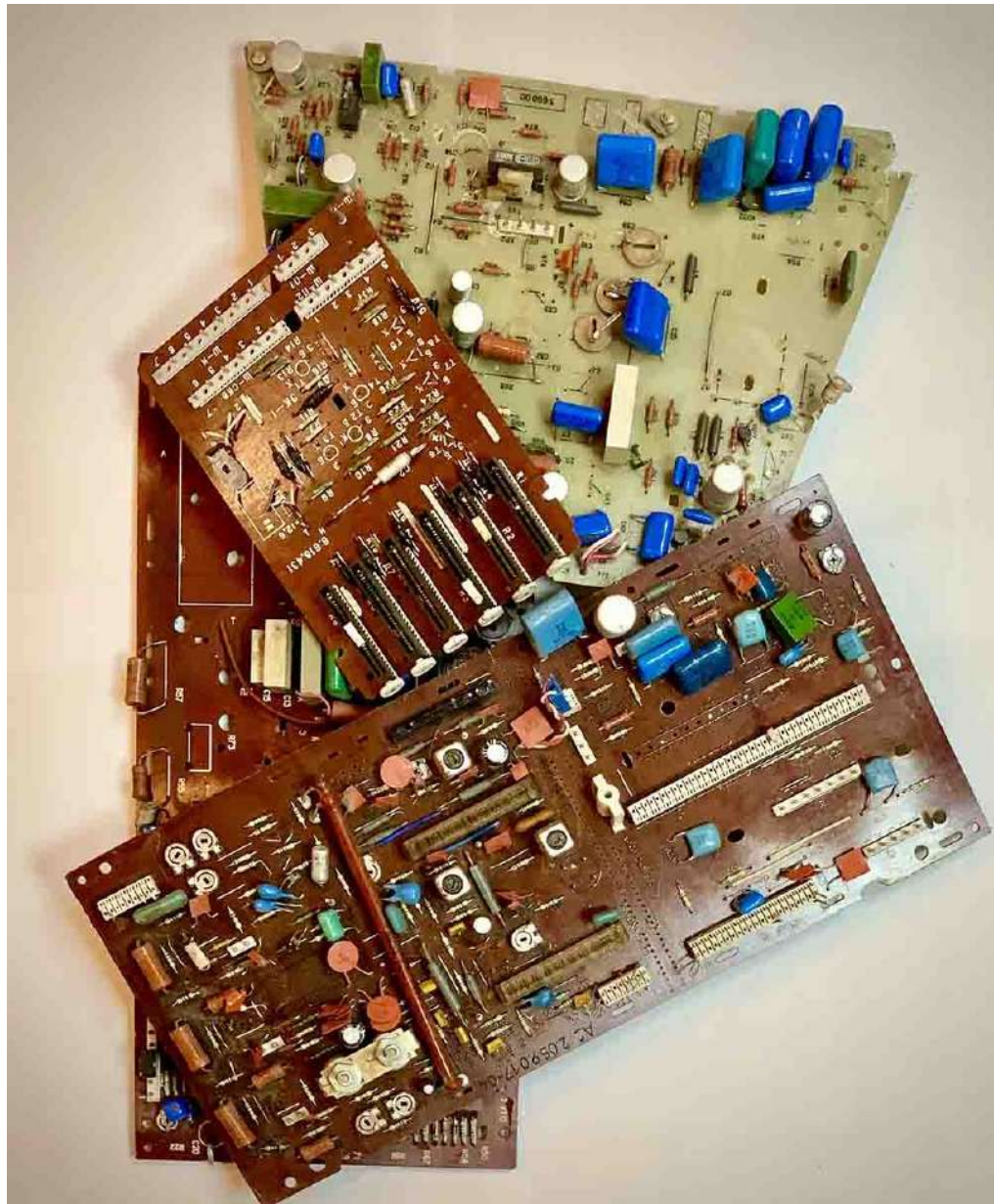




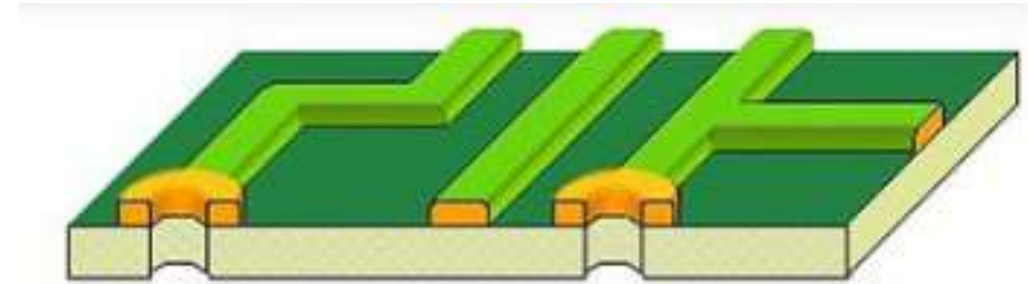
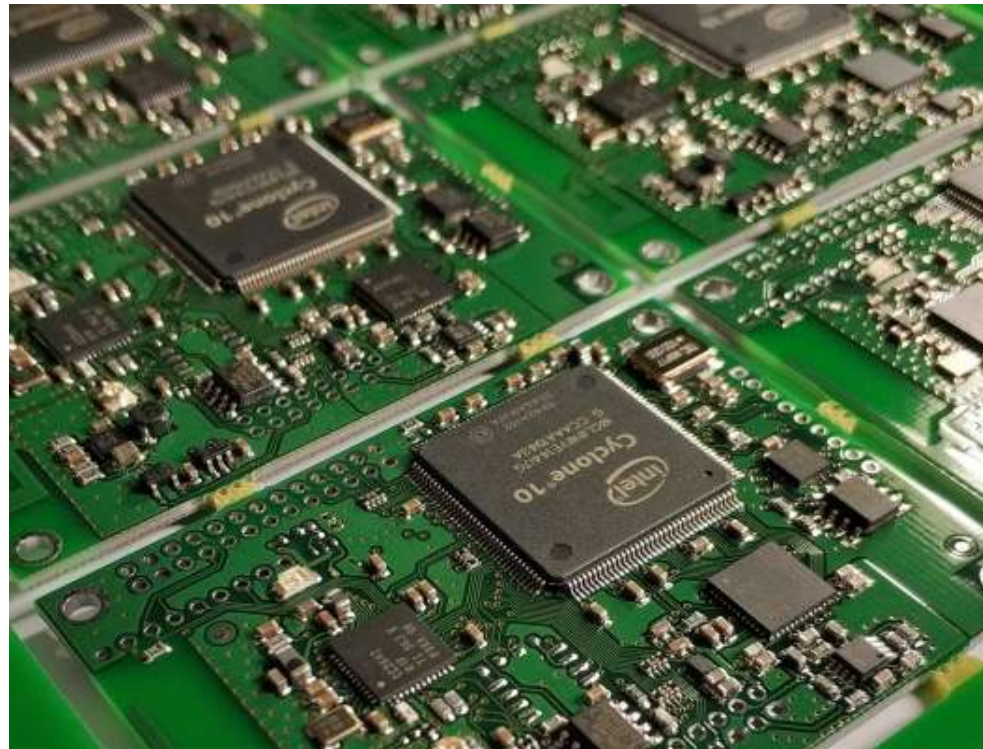
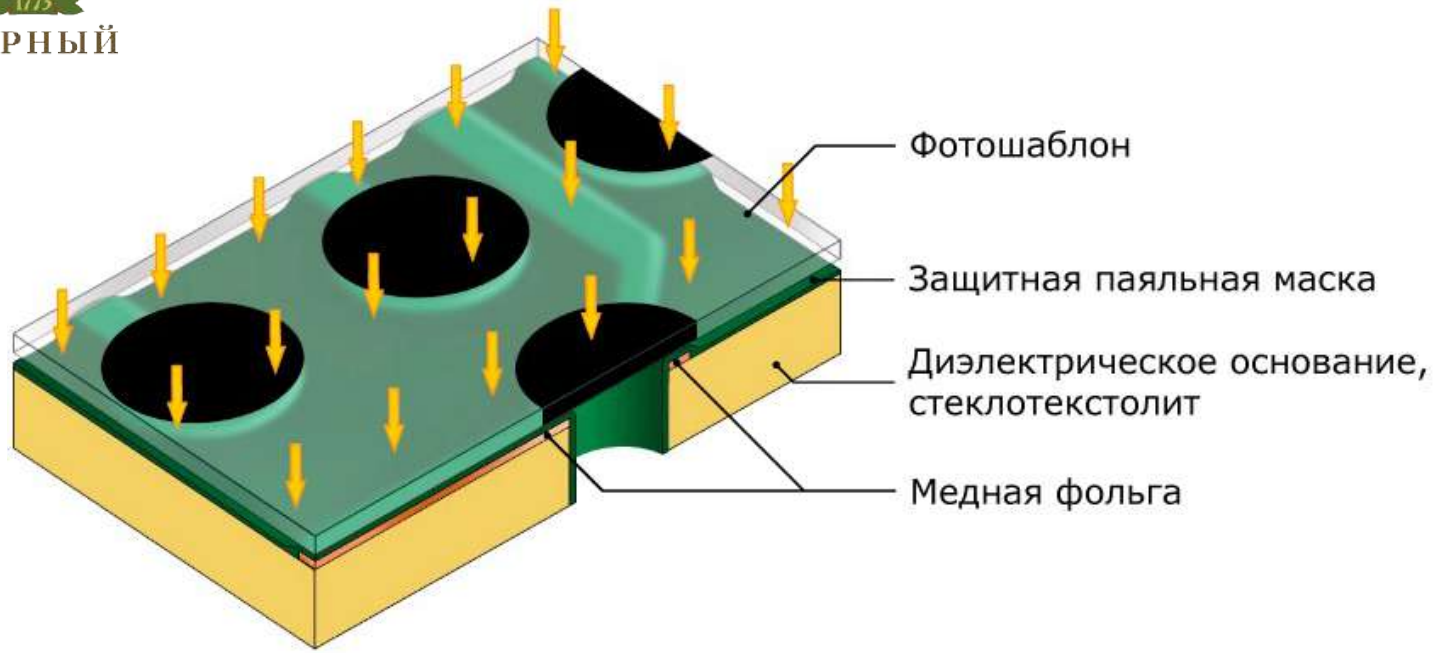
Микроконтроллеры и микрокомпьютеры



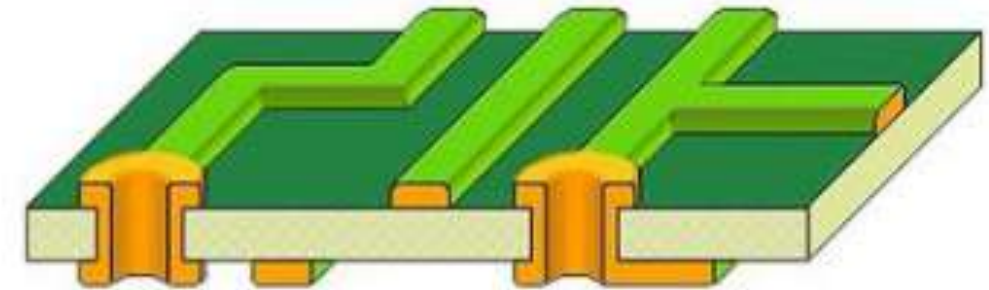
Микроконтроллеры и микрокомпьютеры



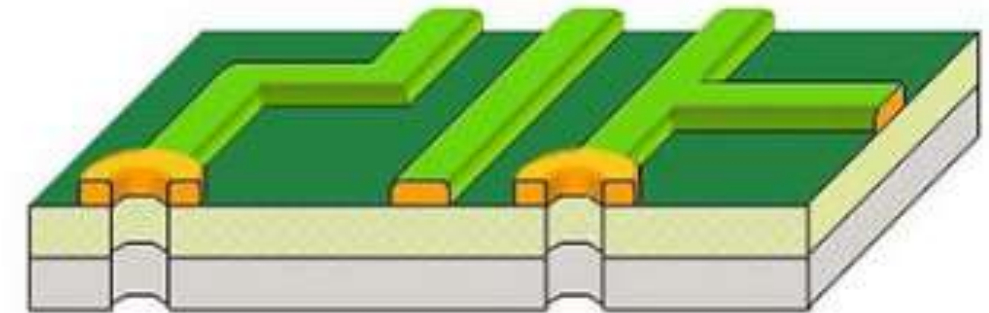
Микроконтроллеры и микрокомпьютеры



Однослойная печатная плата (ОПП)

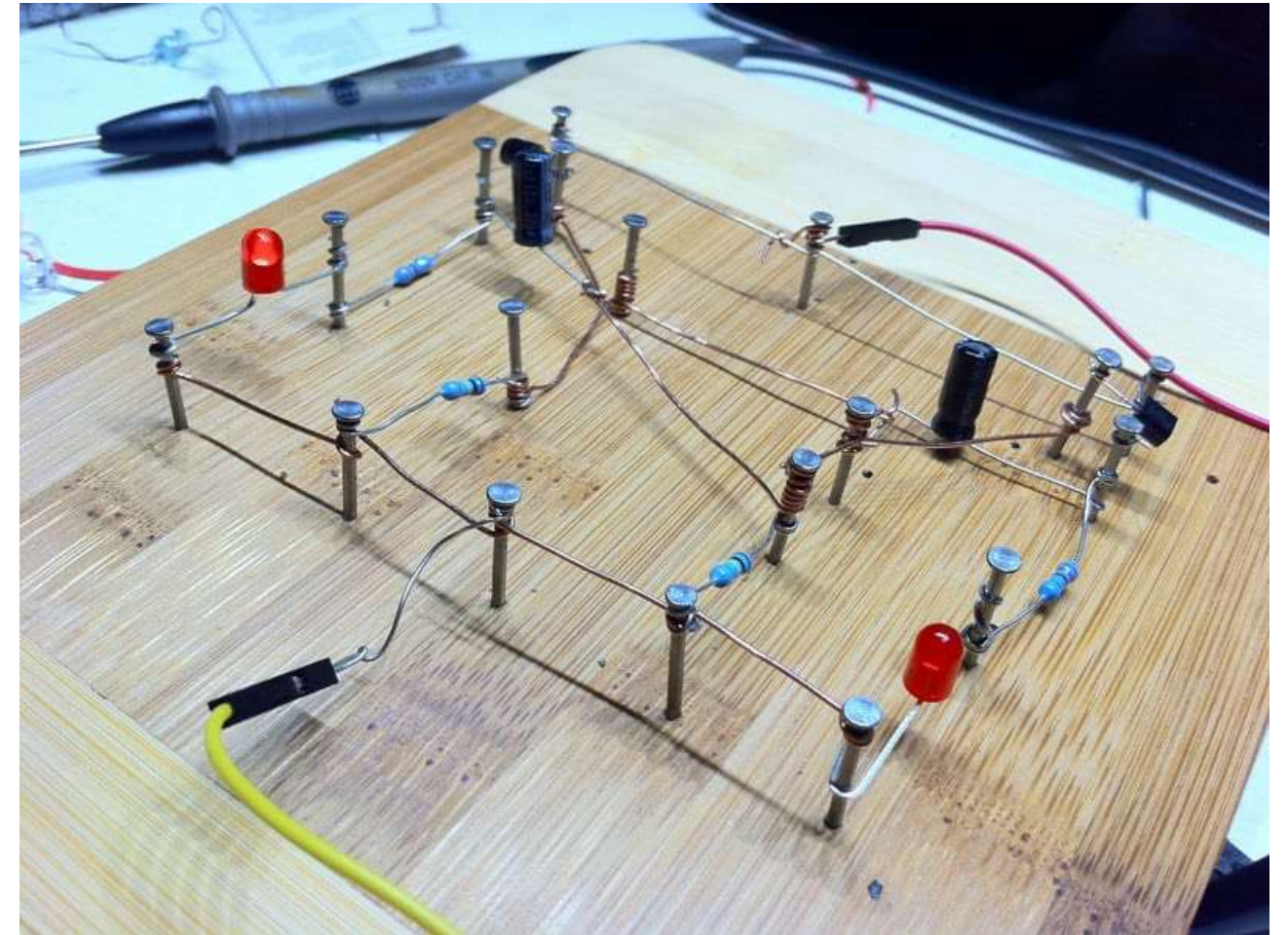


Двухслойная печатная плата (ДПП)

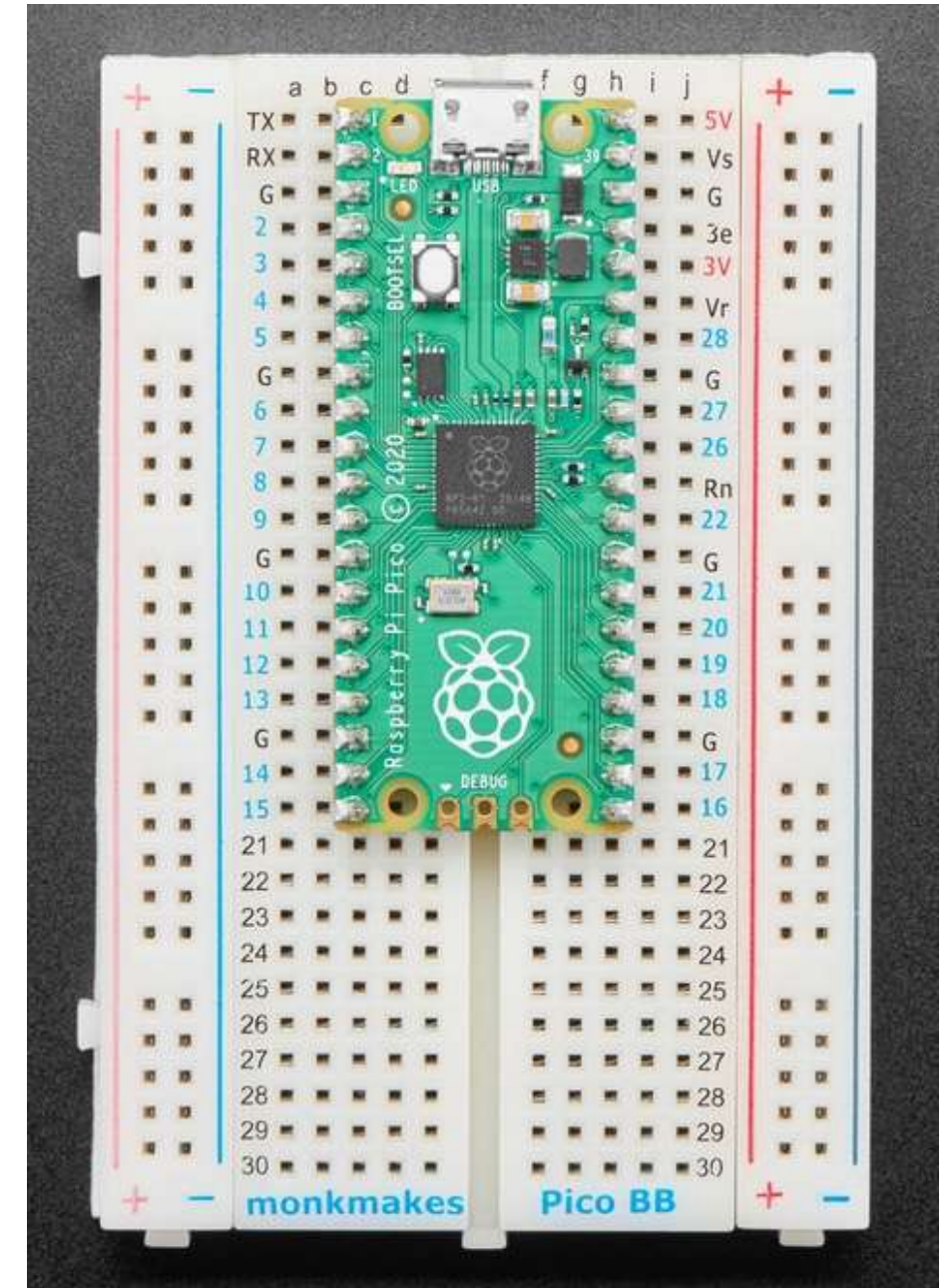
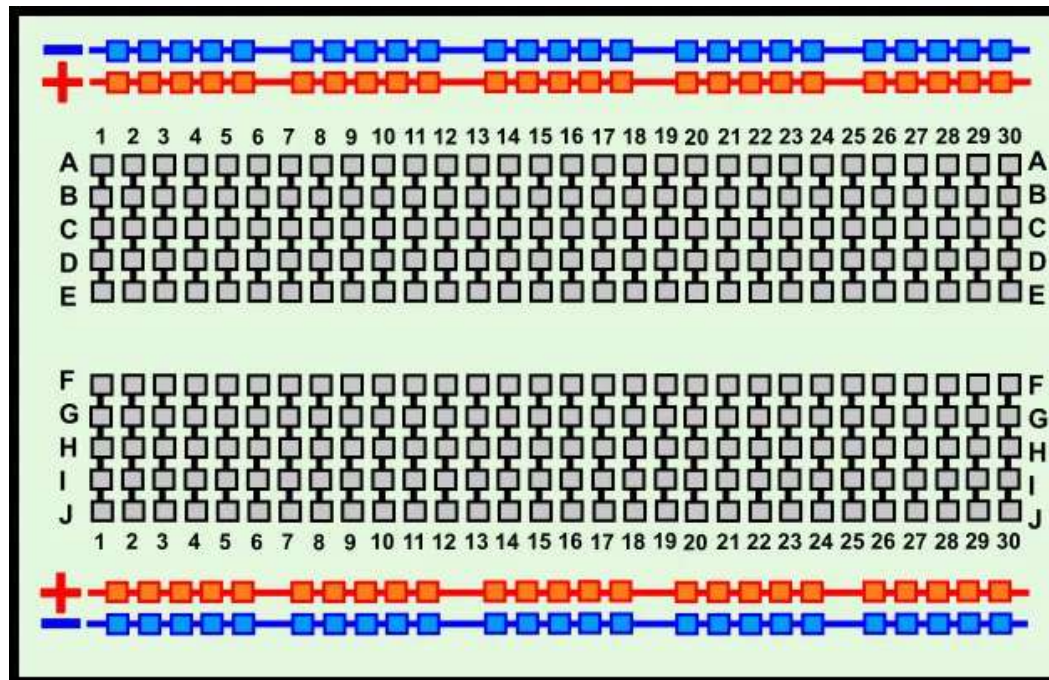
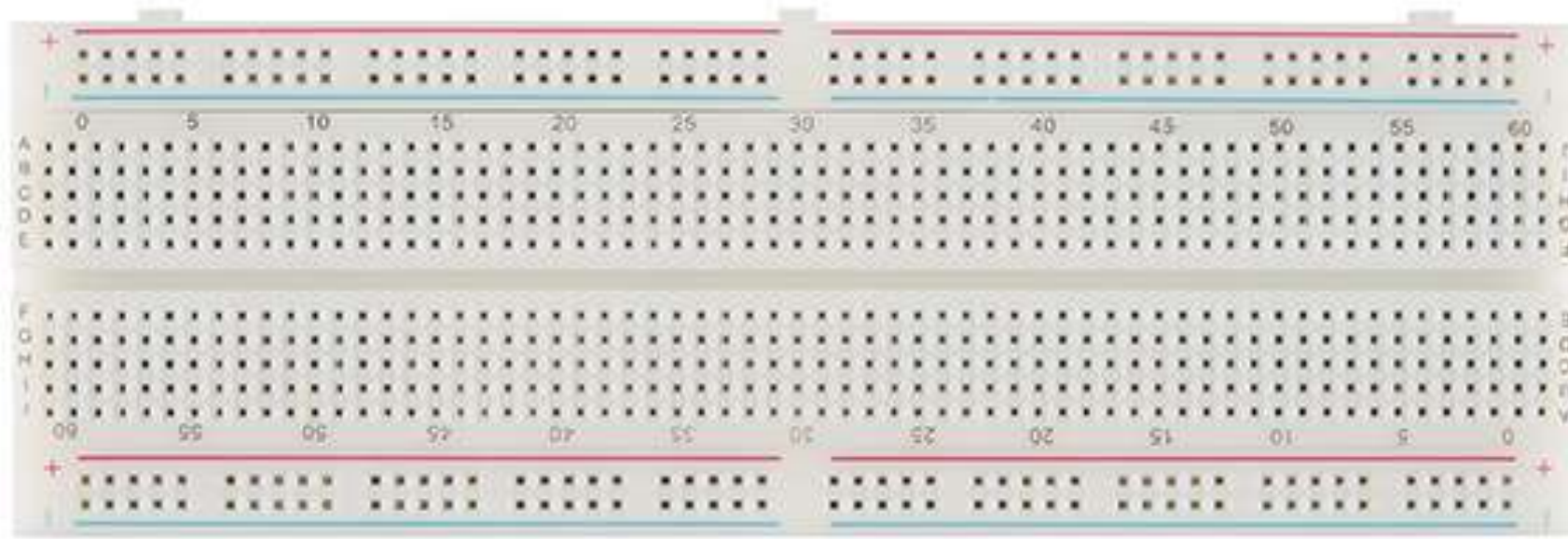


Многослойная печатная плата (МПП)

Микроконтроллеры и микрокомпьютеры

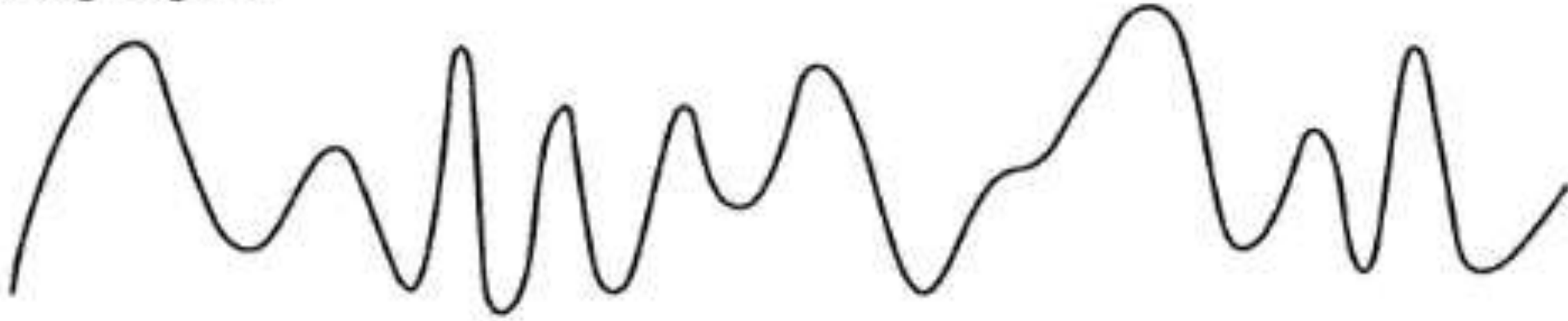


Микроконтроллеры и микрокомпьютеры



Цифровой и аналоговый сигнал

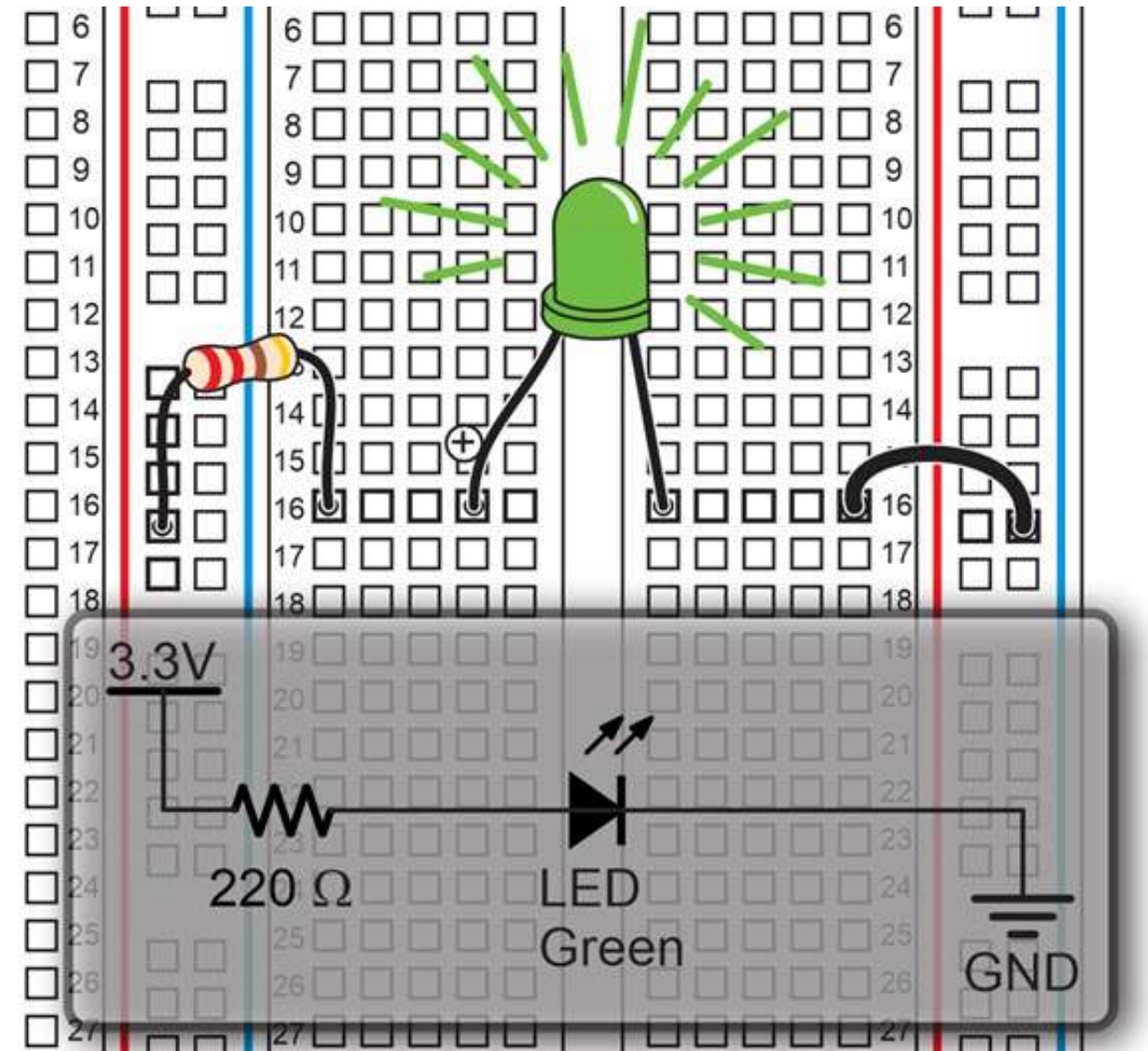
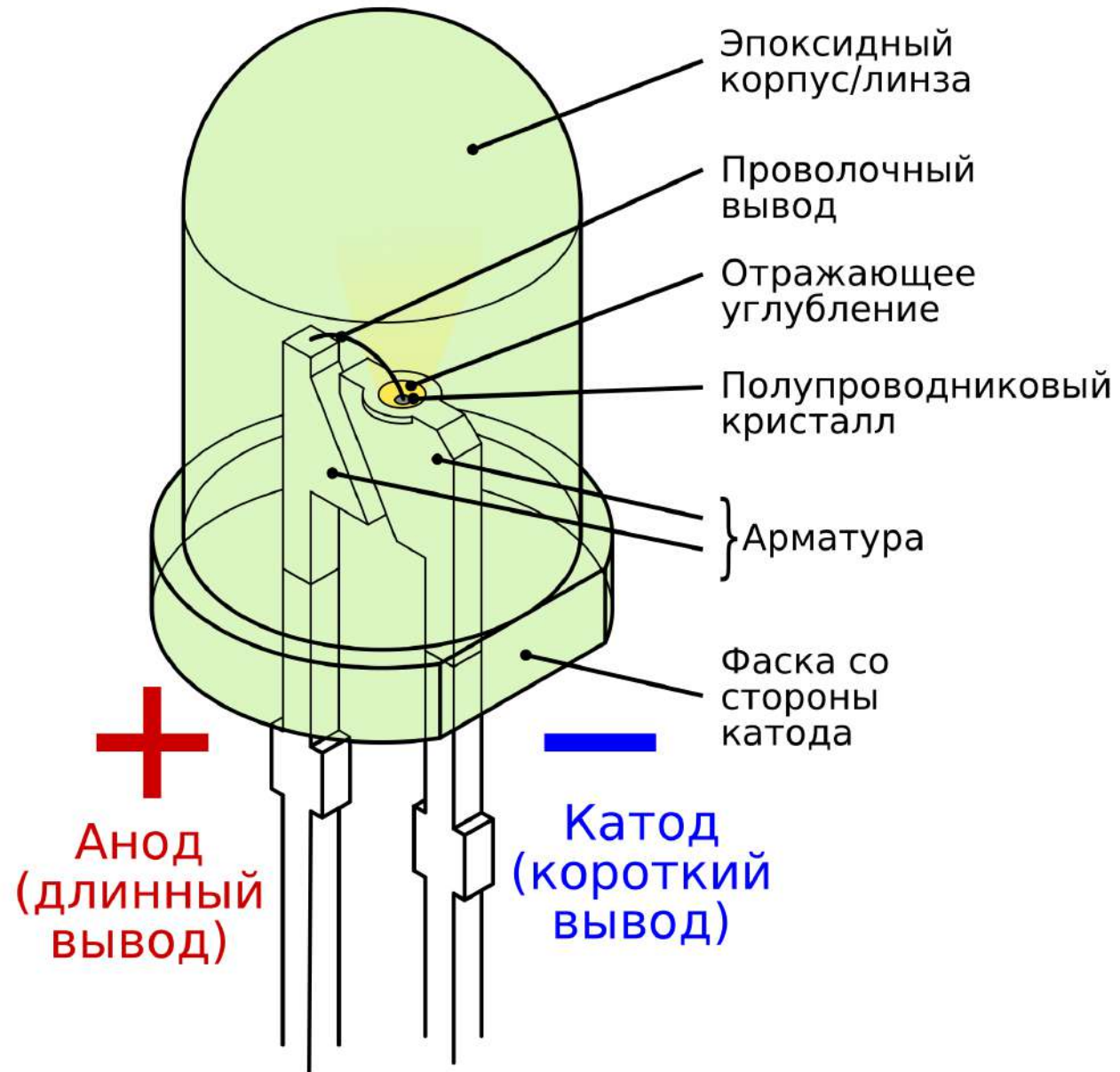
Analog Signal



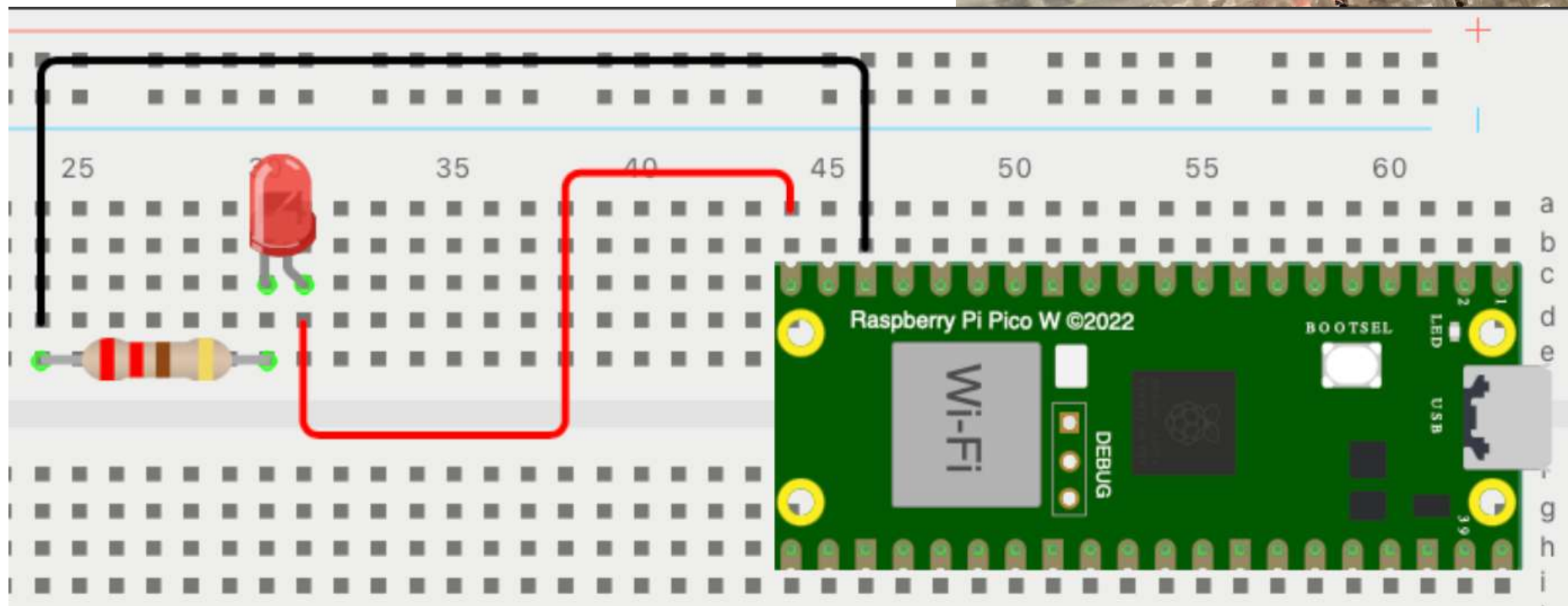
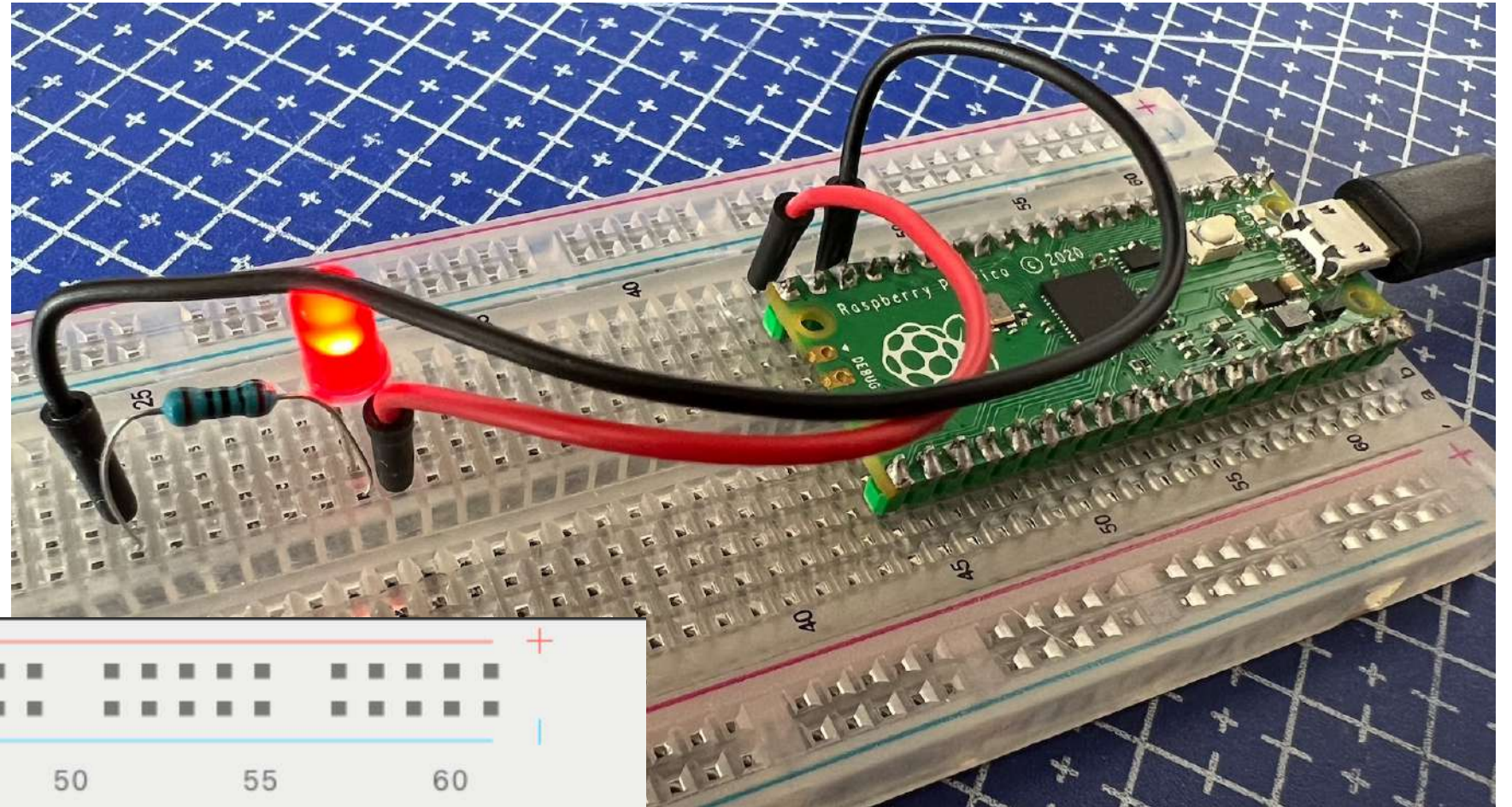
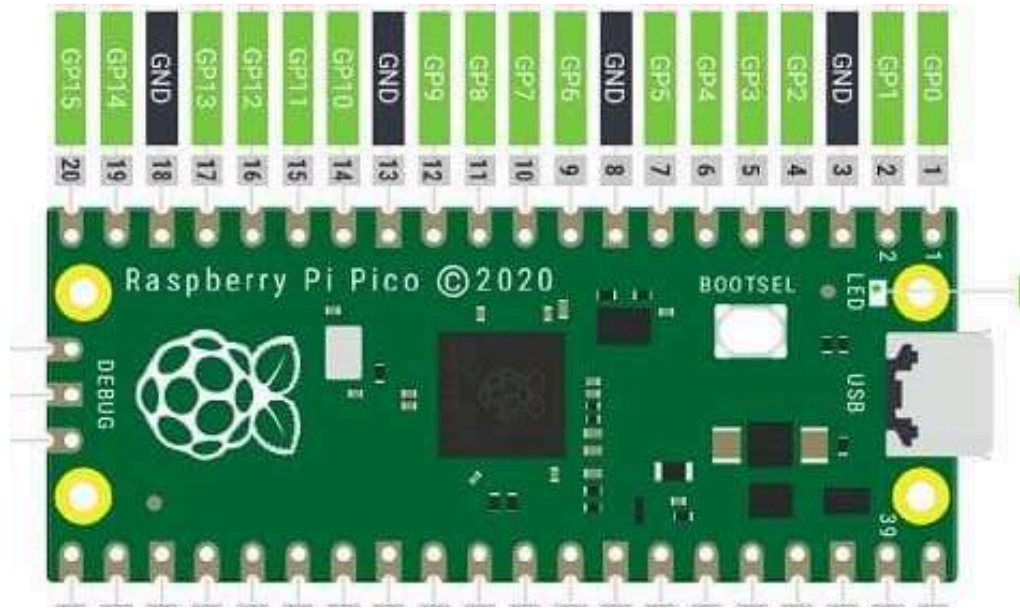
Digital Signal



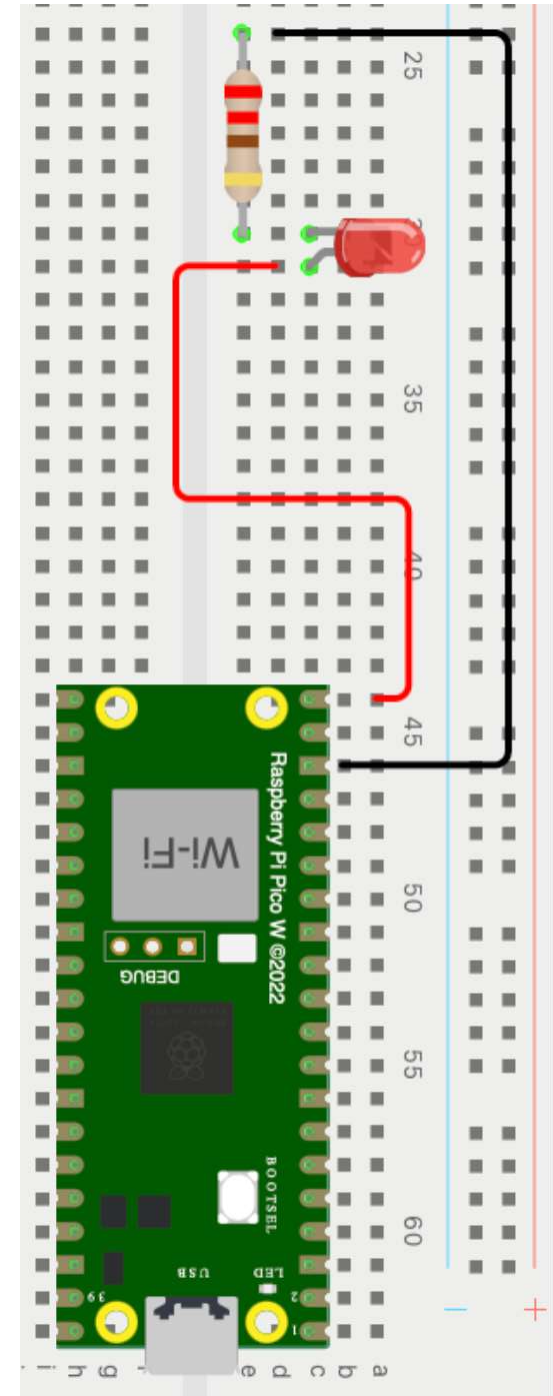
Подключение светодиода



Подключение светодиода



Программирование включения светодиода



```
from machine import Pin
import time
```

```
led_pin = Pin(15, Pin.OUT)
```

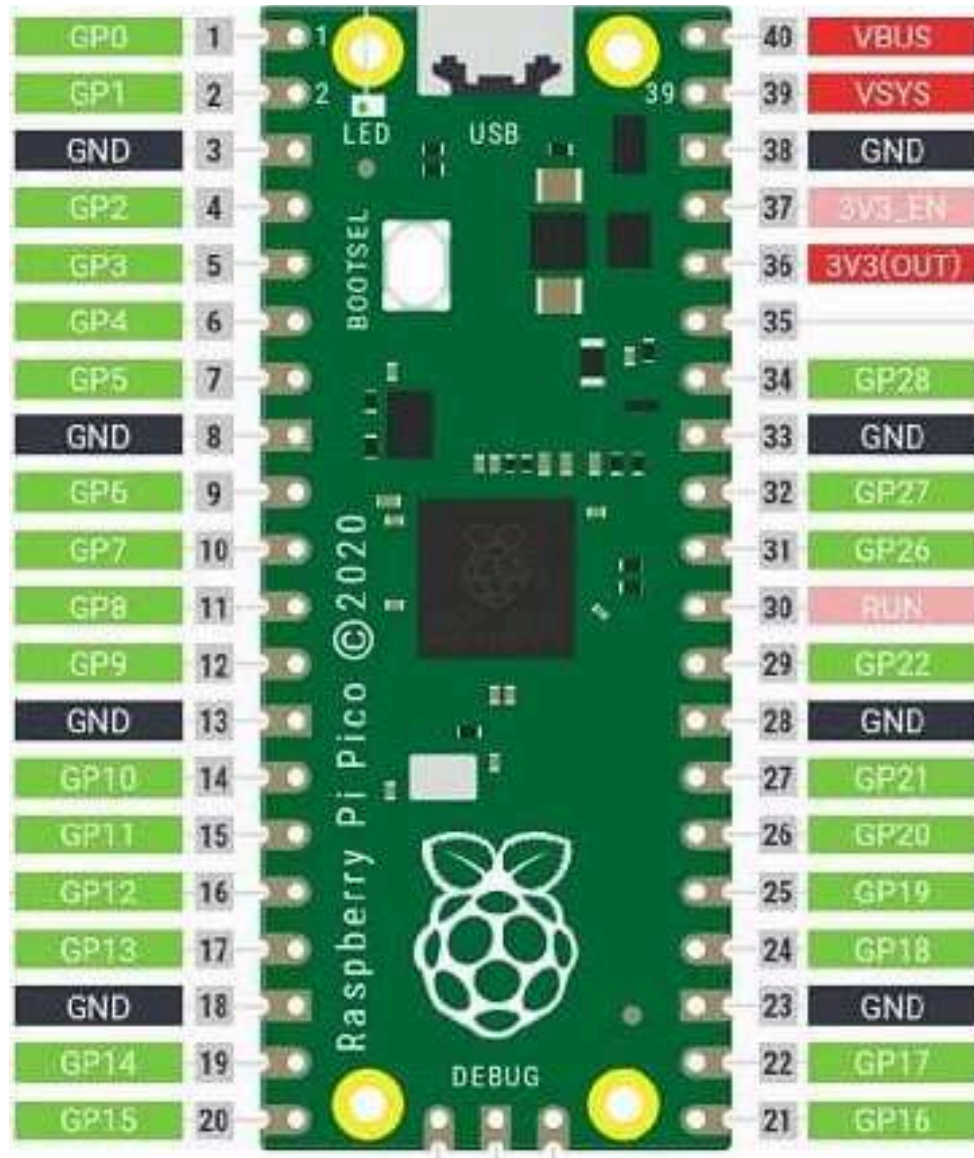
```
while True:
    led_pin.on()
    time.sleep(3)
    led_pin.off()
    time.sleep(1)
```

```
while True:
    led_pin.high()
    time.sleep(3)
    led_pin.low()
    time.sleep(1)
```

```
while True:
    led_pin.value(1)
    time.sleep(3)
    led_pin.value(0)
    time.sleep(1)
```

```
counter = 0
while True:
    led_pin.value(counter % 2)
    time.sleep(3)
    counter += 1
```

Чтение цифрового сигнала

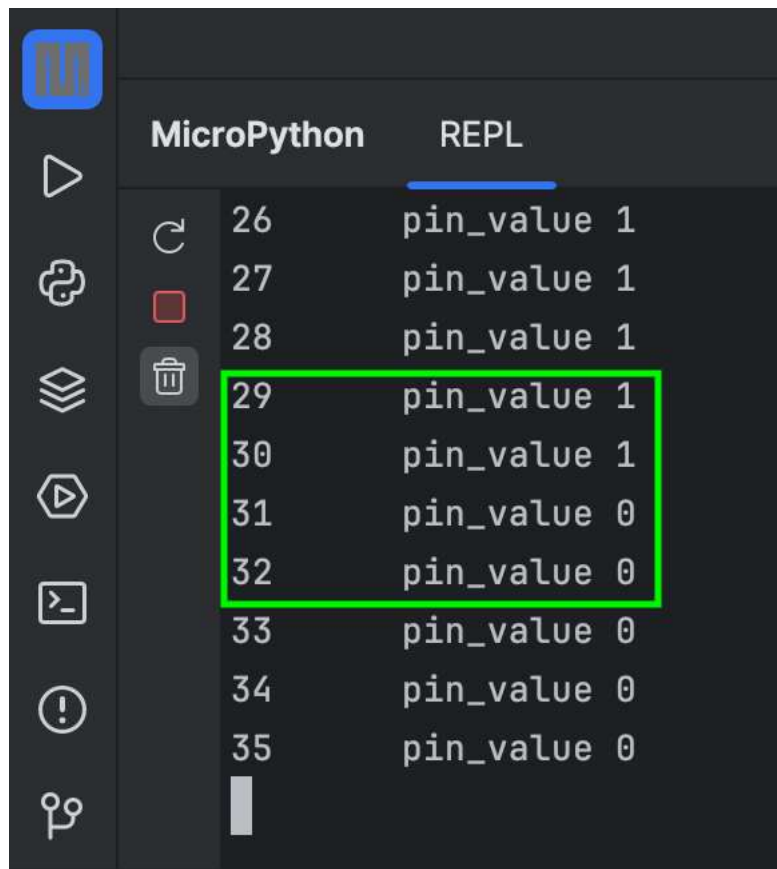


```
from machine import Pin
import time

read_pin = Pin(16, Pin.IN)

counter = 0
while True:
    pin_value = read_pin.value()
    print(counter, "\tpin_value", pin_value)
    time.sleep(0.2)
    counter += 1
```

Чтение цифрового сигнала



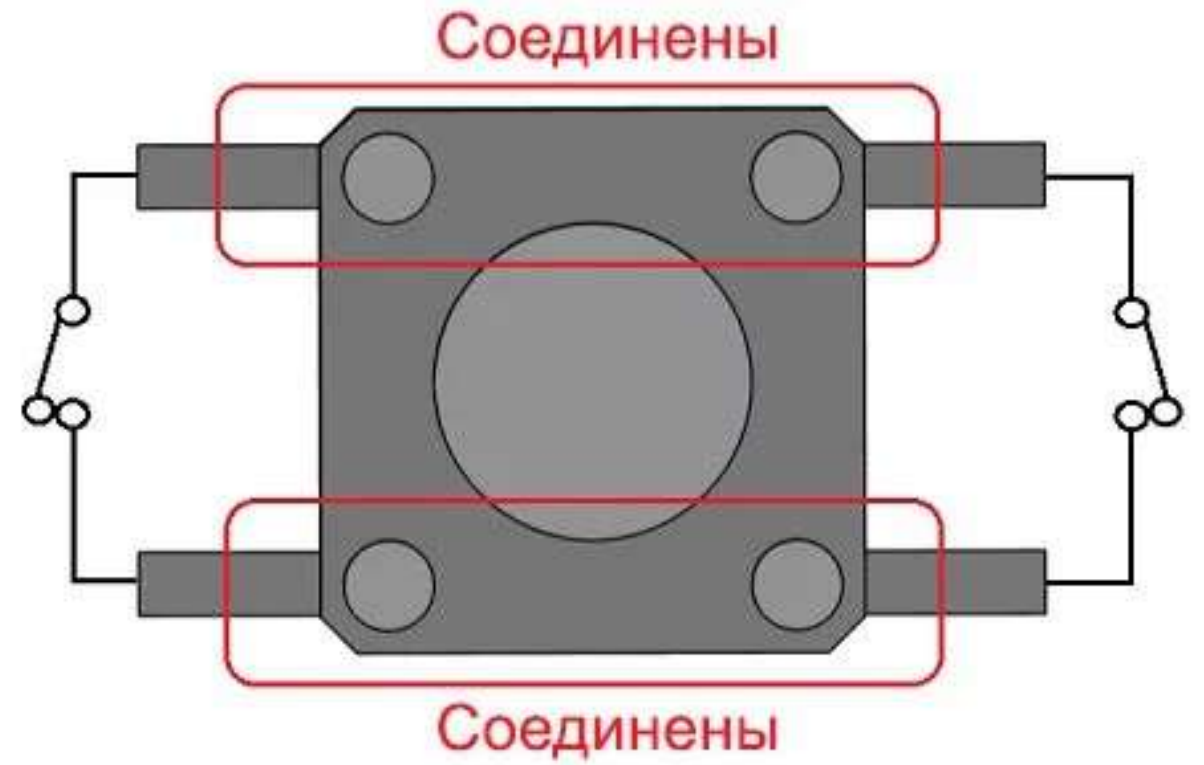
Line	pin_value
26	1
27	1
28	1
29	1
30	1
31	0
32	0
33	0
34	0
35	0

```
from machine import Pin
import time

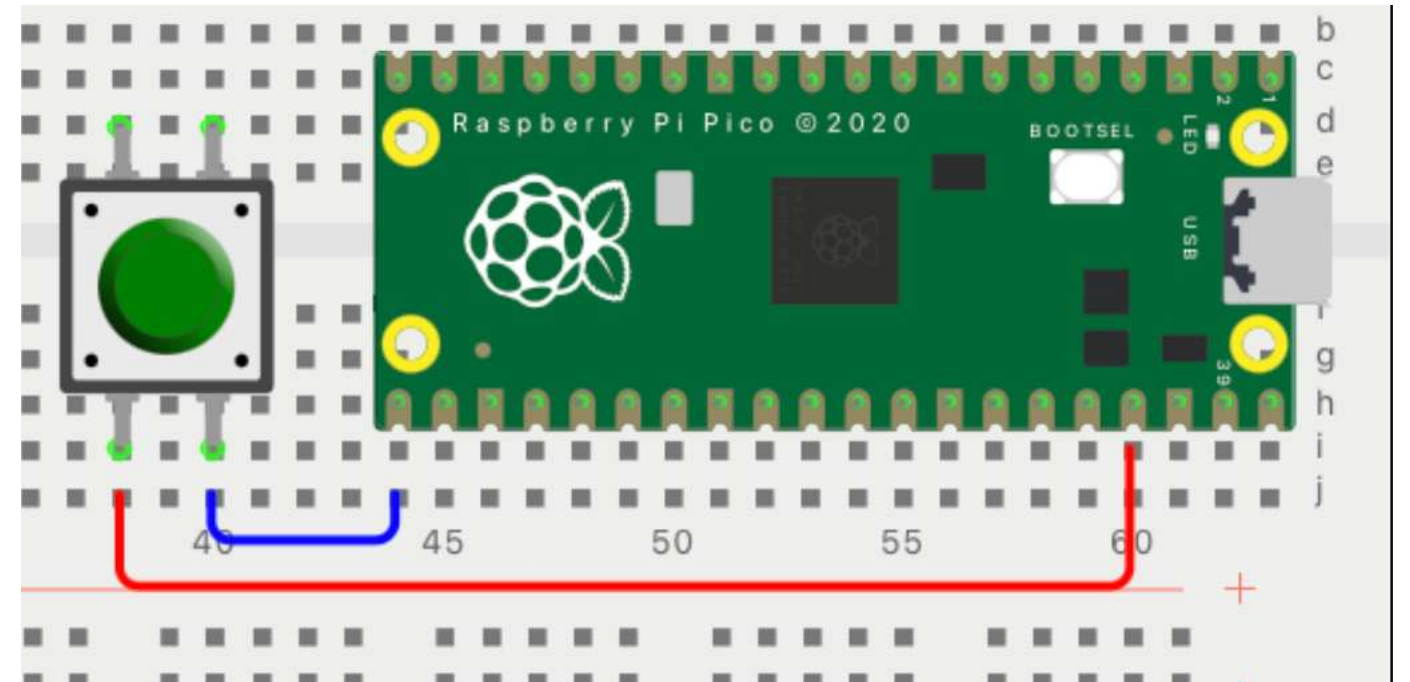
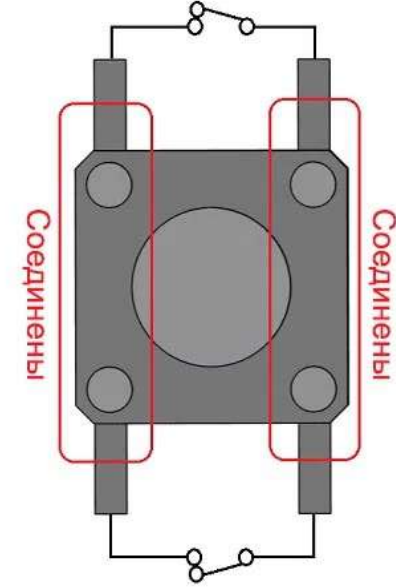
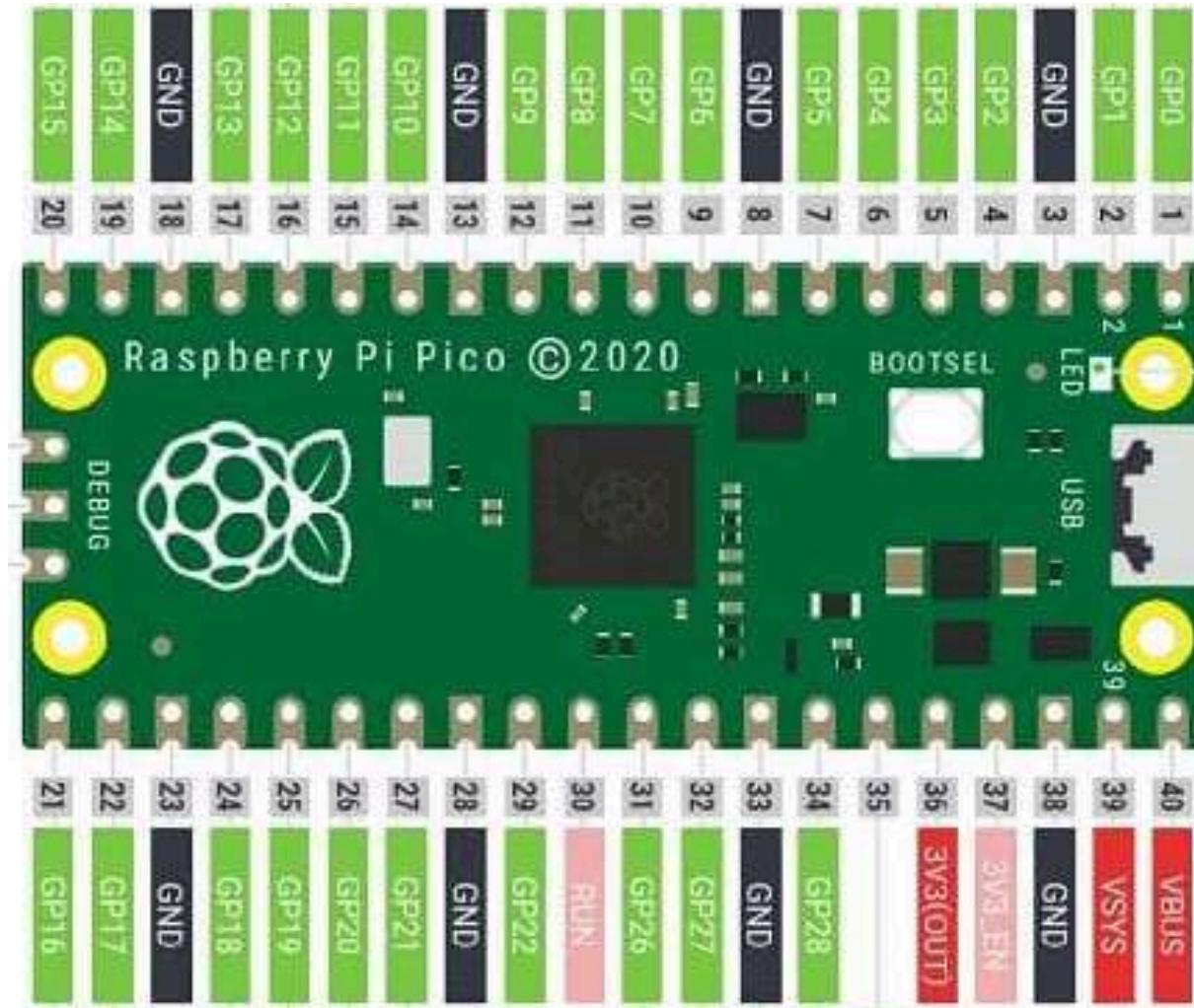
read_pin = Pin(16, Pin.IN)

counter = 0
while True:
    pin_value = read_pin.value()
    print(counter, "\tpin_value", pin_value)
    time.sleep(0.2)
    counter += 1
```

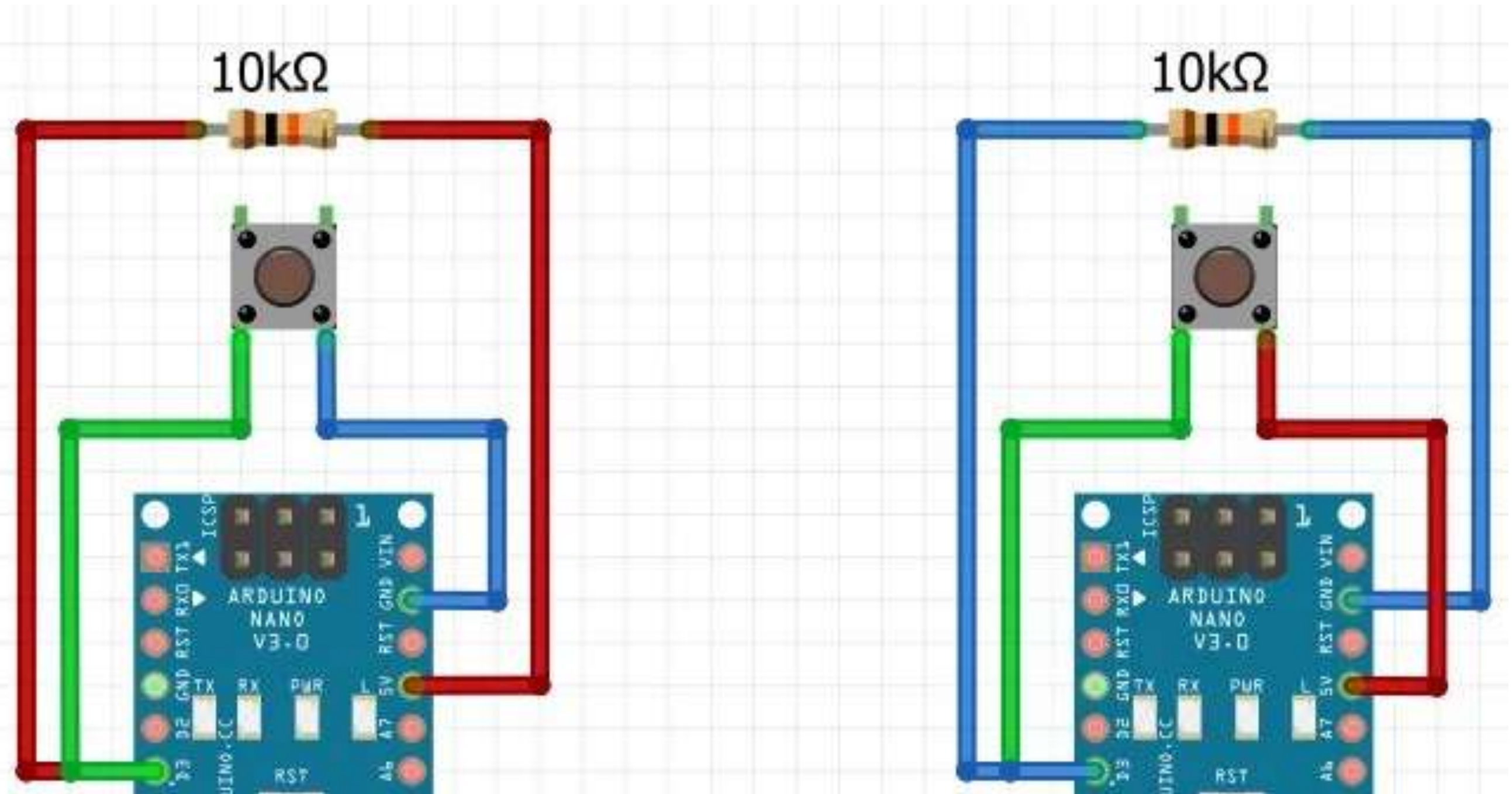
Тактовая кнопка



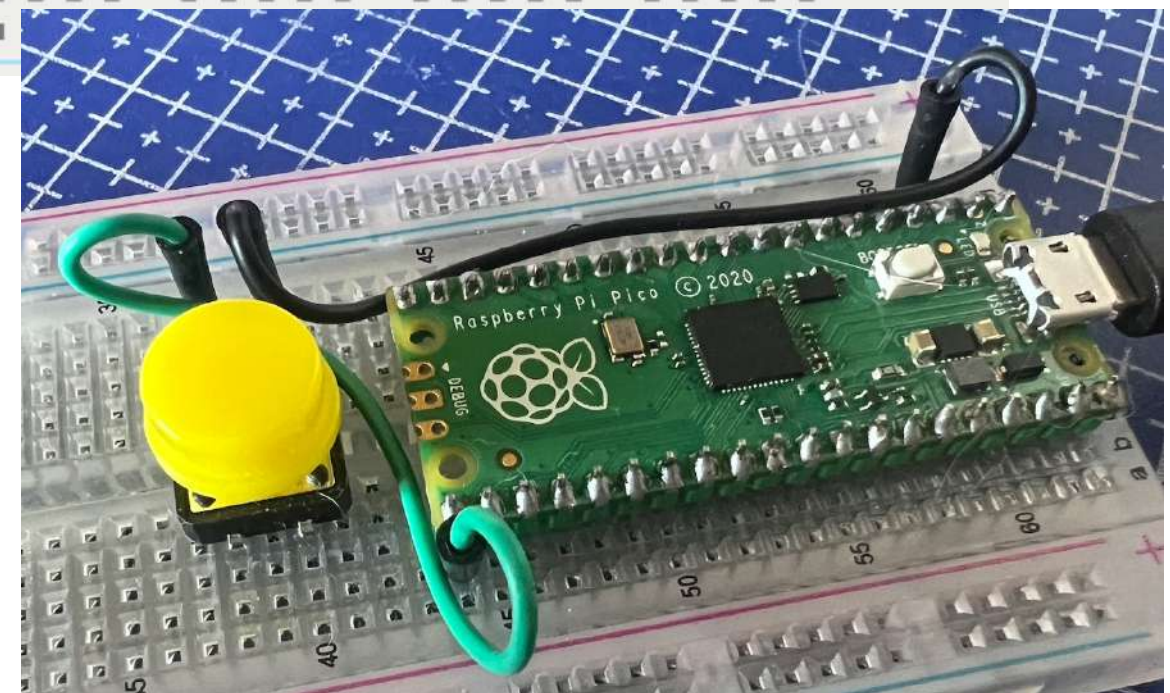
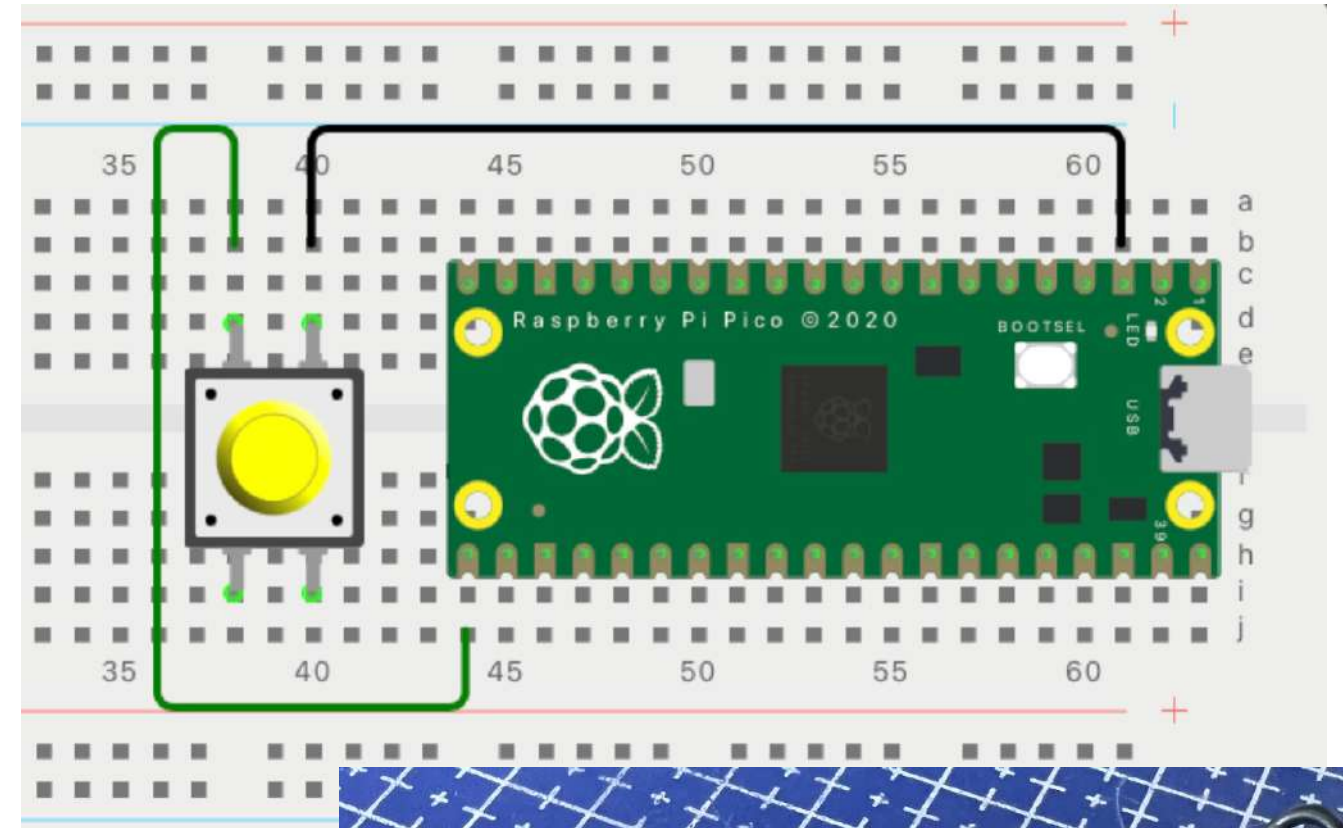
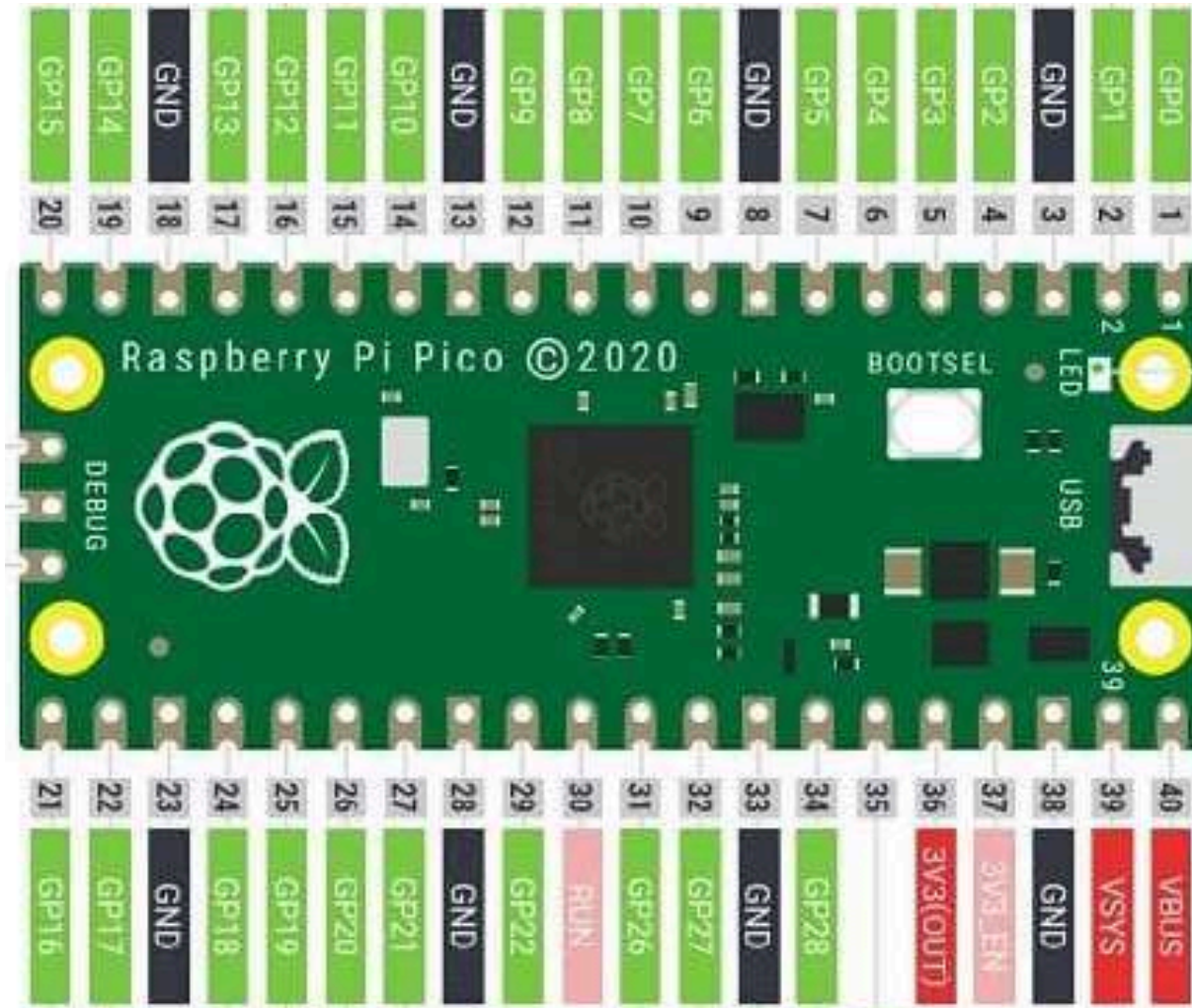
Опасное подключение тактовой кнопки



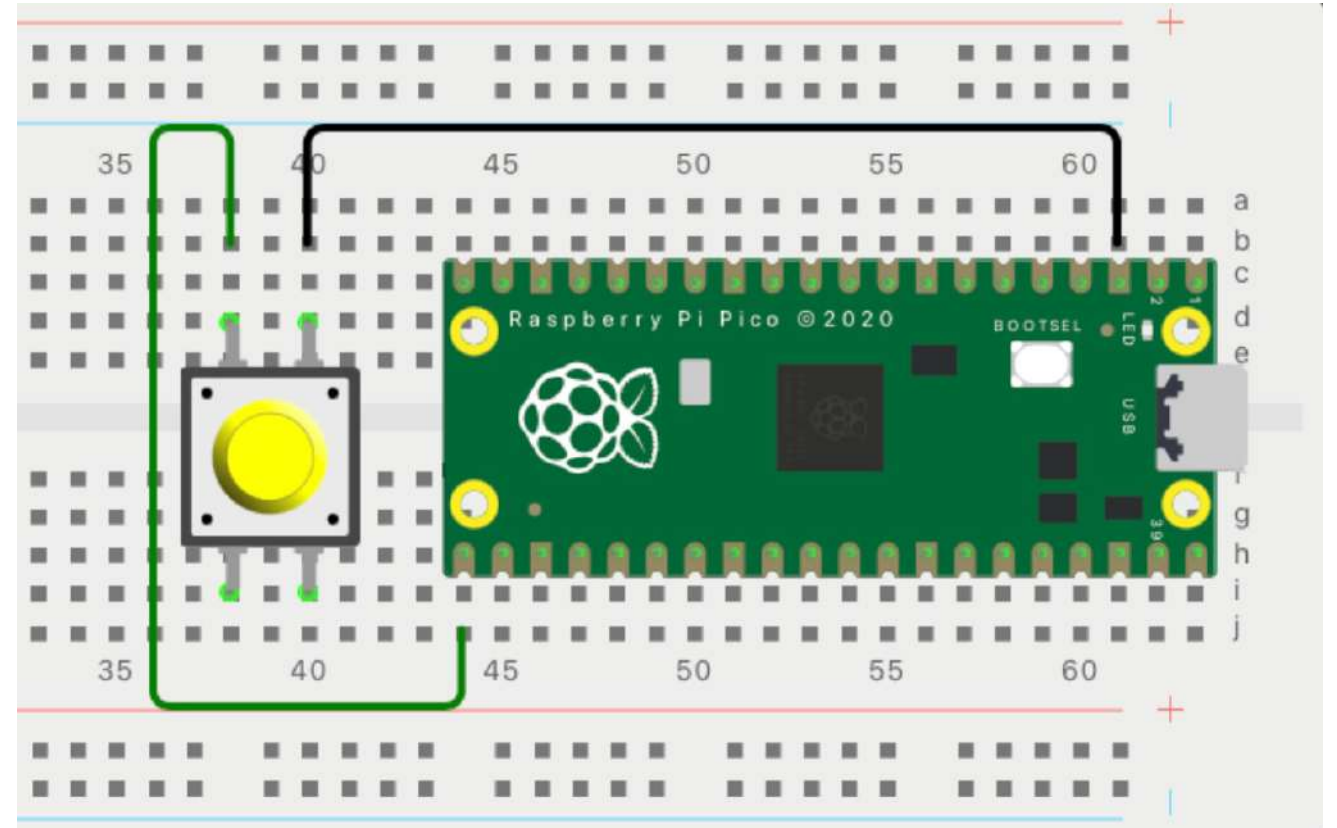
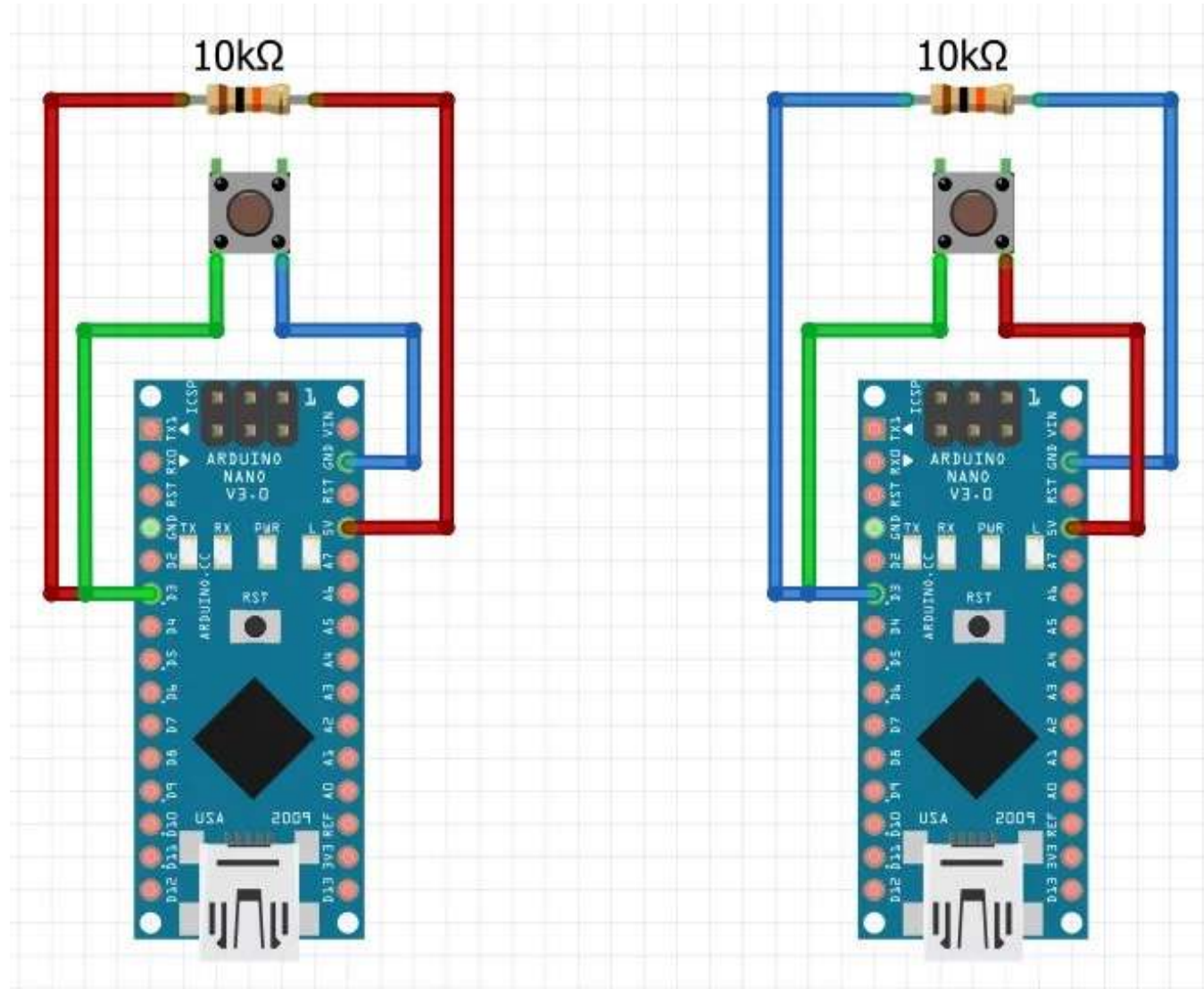
Подтяжка сигнала



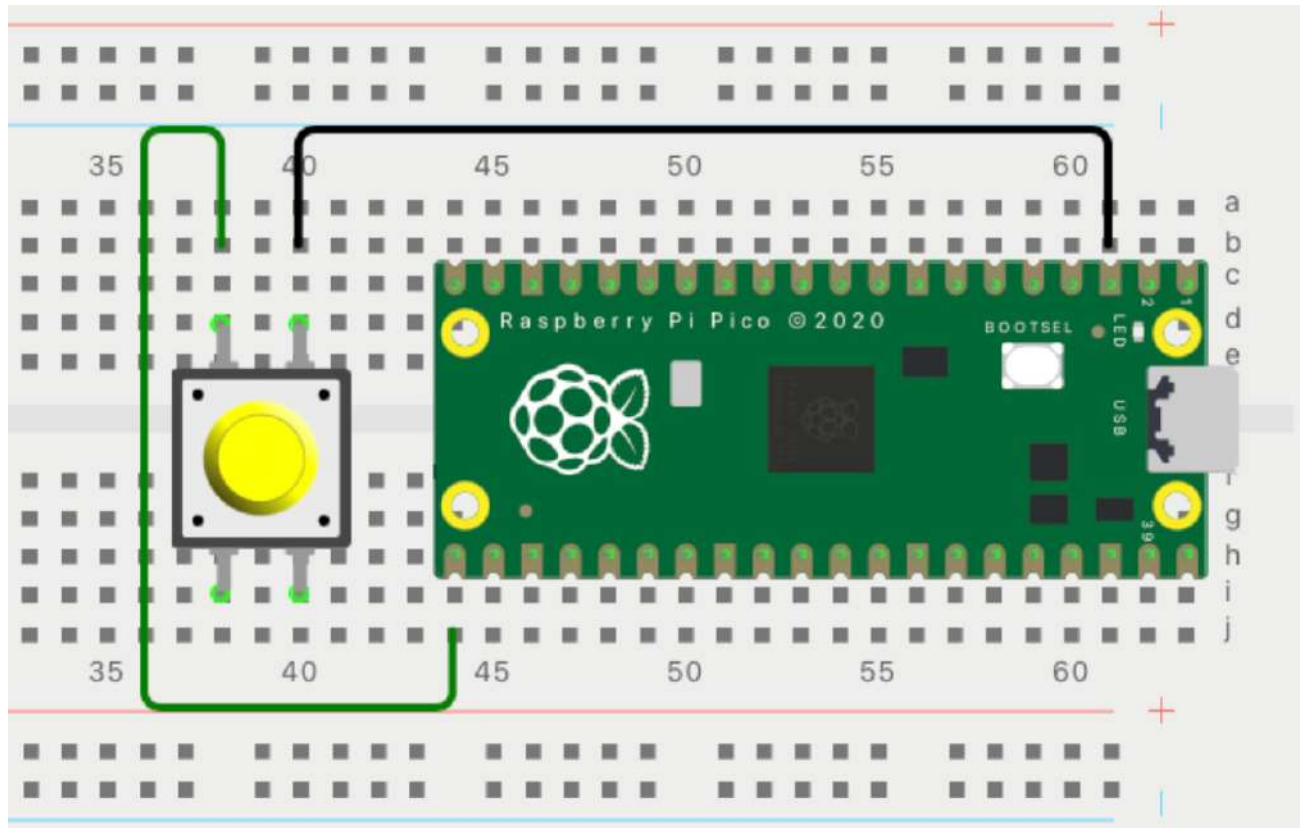
Опасное подключение тактовой кнопки



Схемы подключения тактовой кнопки



Подтяжка кнопки к встроенному резистору

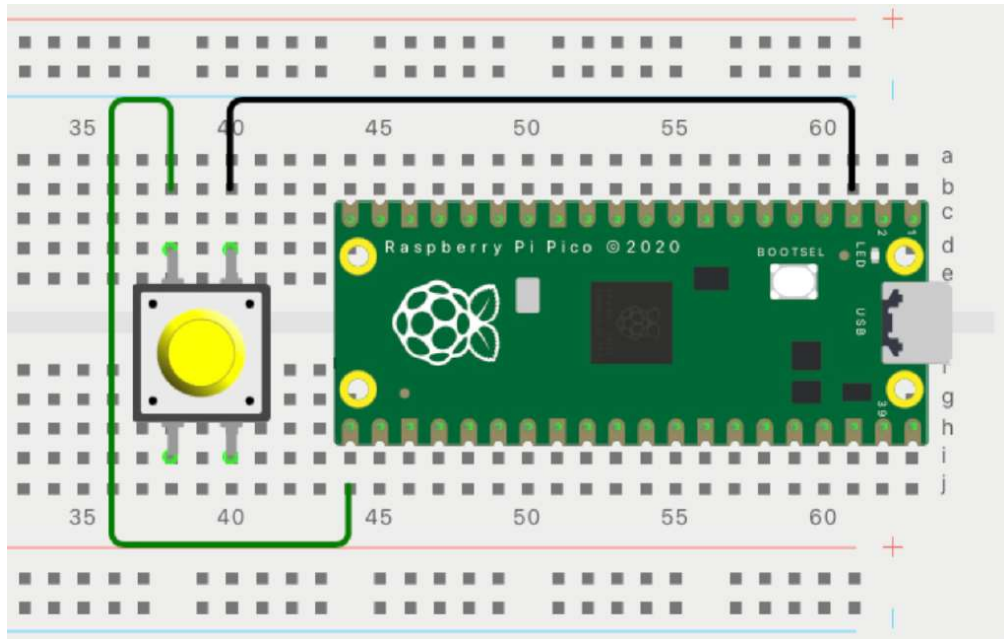


```
from machine import Pin
import time

button_pin = 16
button = Pin(button_pin, Pin.IN, Pin.PULL_UP)

counter = 0
while True:
    button_value = button.value()
    print(counter, "\tbutton_state", button_value)
    time.sleep(0.2)
    counter += 1
```

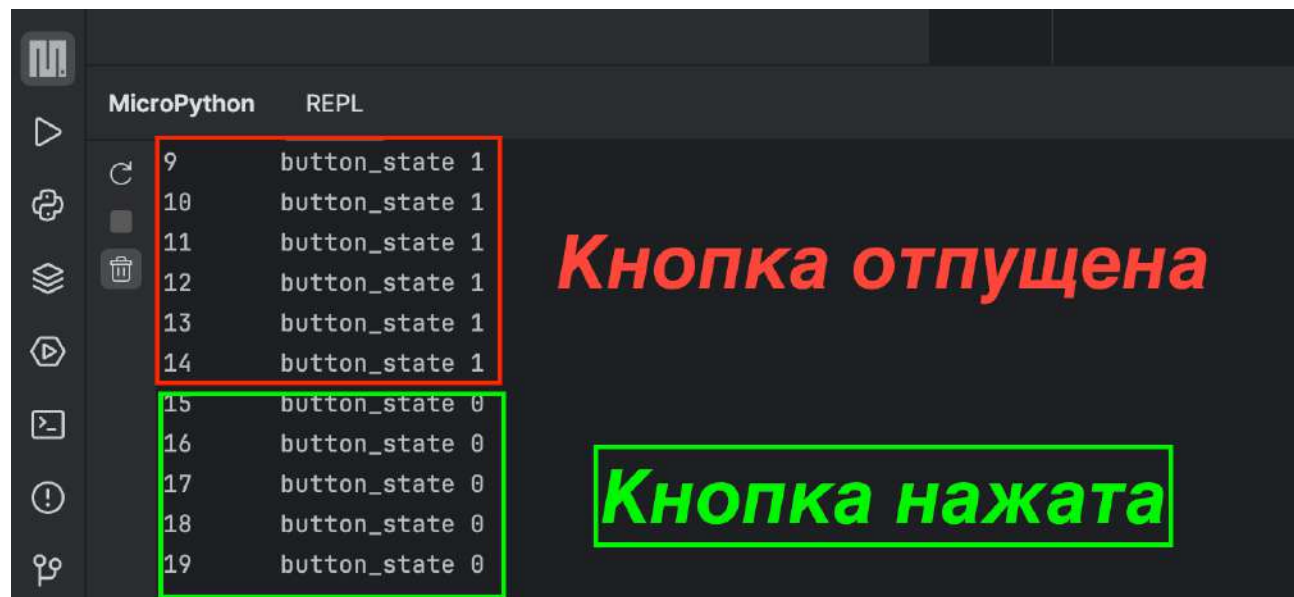
Программирование тактовой кнопки



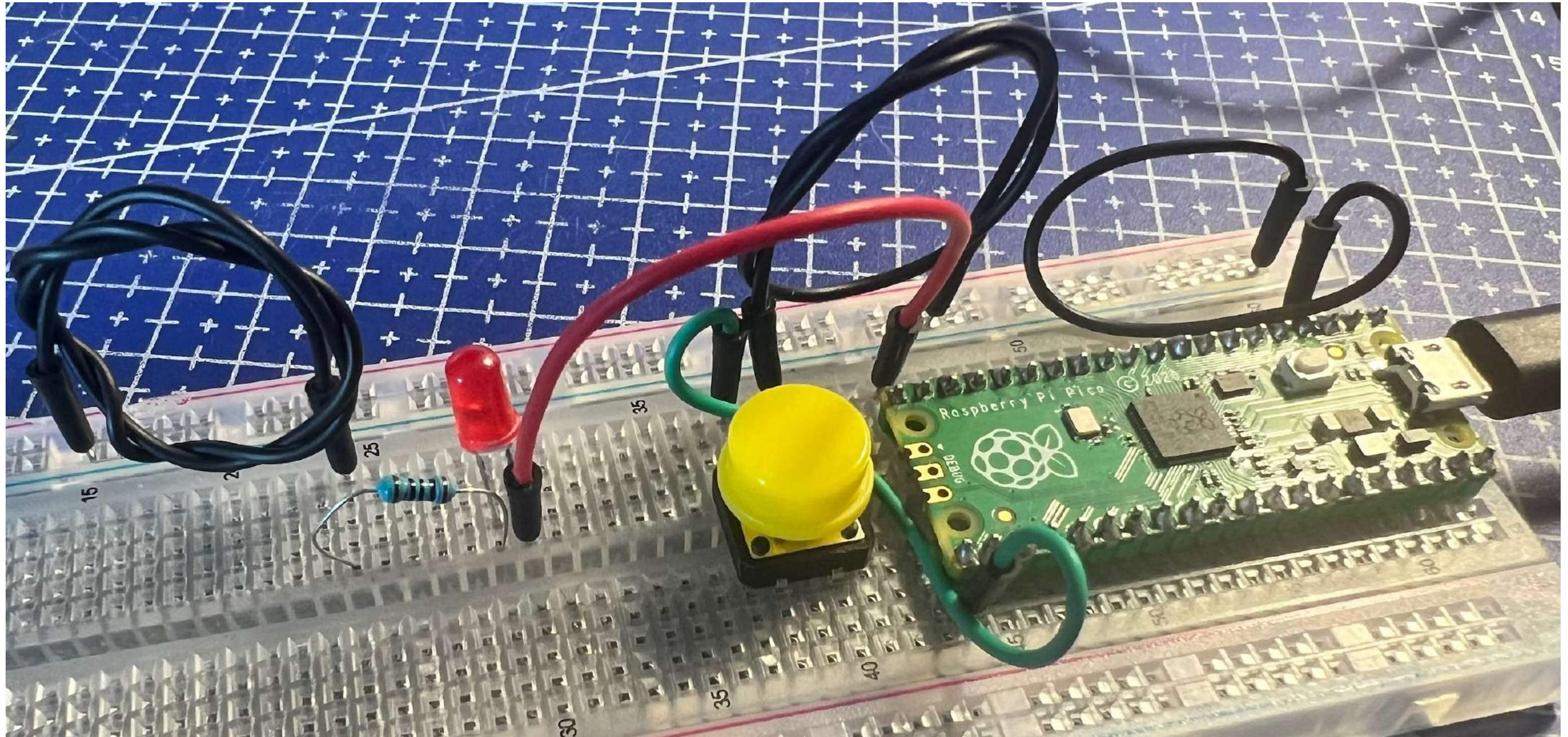
```
from machine import Pin
import time

button_pin = 16
button = Pin(button_pin, Pin.IN, Pin.PULL_UP)

counter = 0
while True:
    button_value = button.value()
    print(counter, "\tbutton_state", button_value)
    time.sleep(0.2)
    counter += 1
```

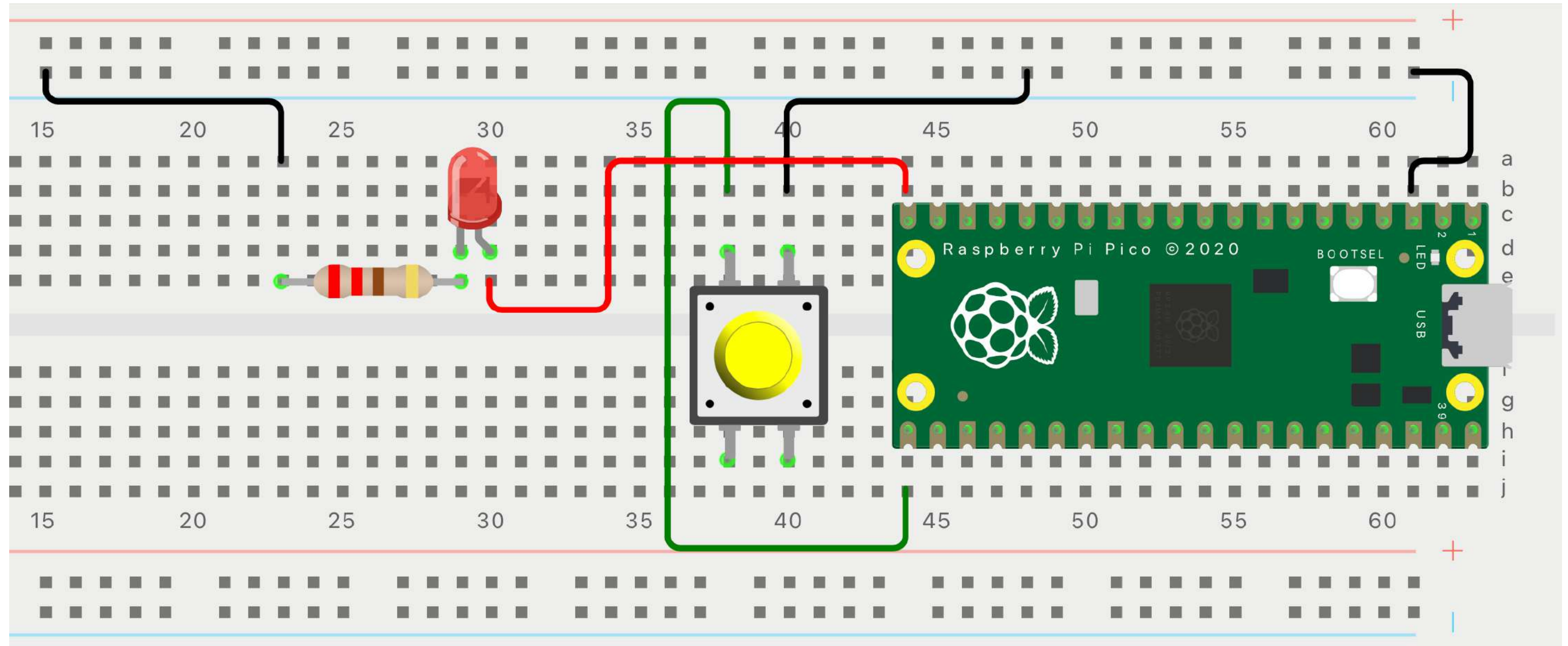


Общая схема подключения





Общая схема подключения





Общий программный код

```
from machine import Pin
import time

red_led_pin = 15
button_pin = 16

red_led = Pin(red_led_pin, Pin.OUT)
button = Pin(button_pin, Pin.IN, Pin.PULL_UP)

while True:
    button_value = button.value()
    red_led.value(button_value)
    time.sleep(0.2)
```



Общий программный код

```
from machine import Pin
import time

red_led_pin = 15
button_pin = 16

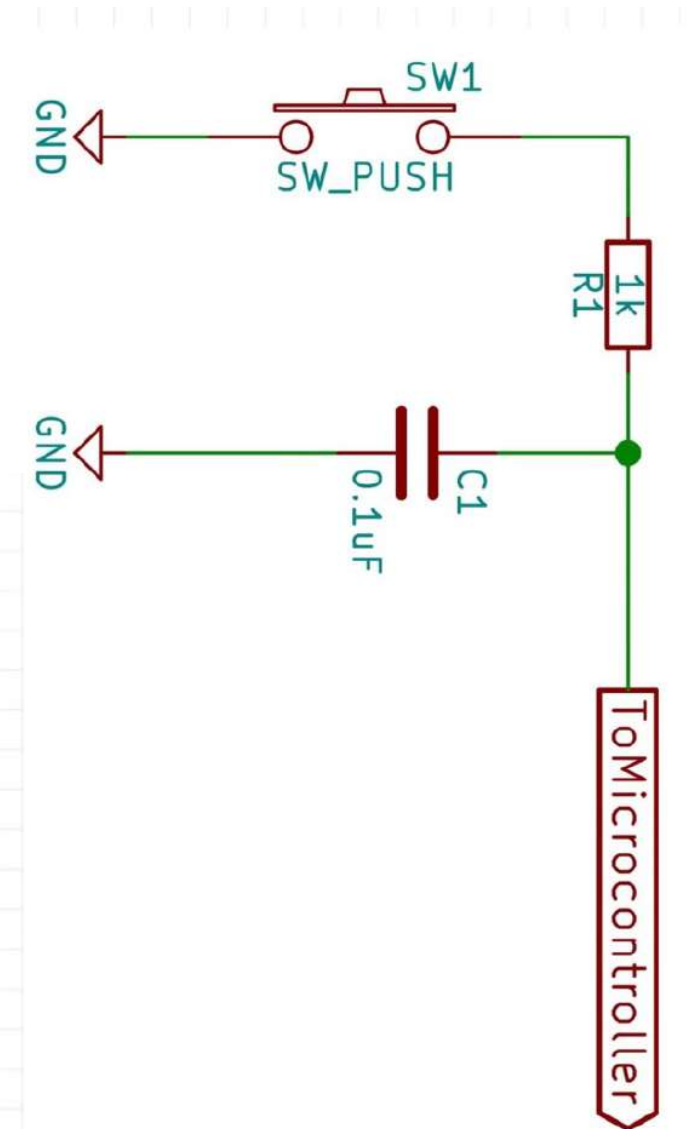
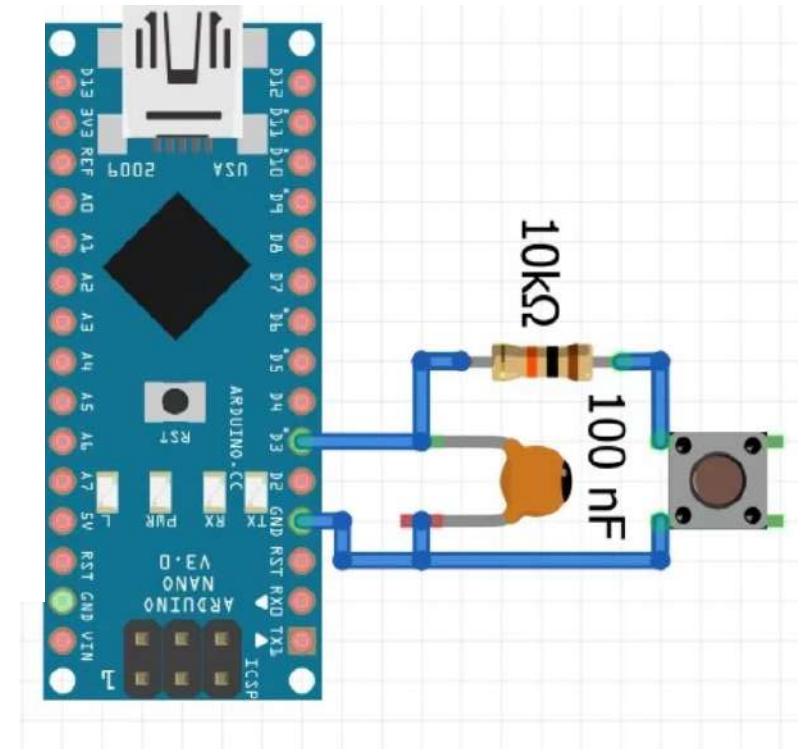
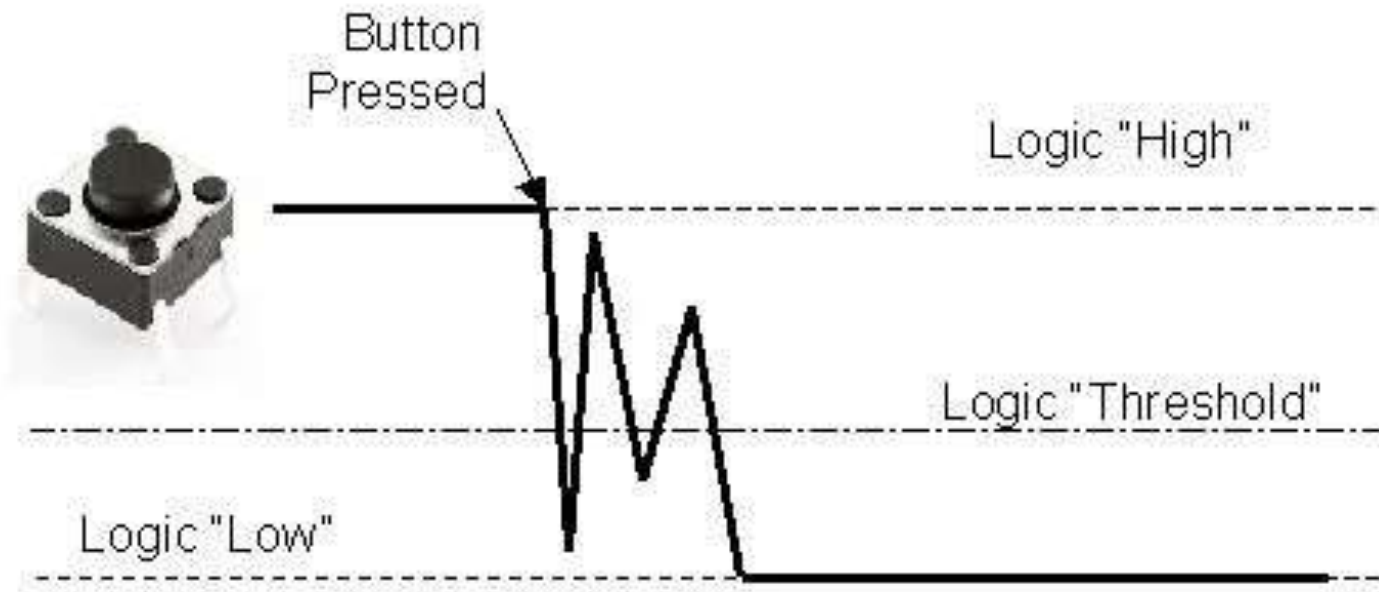
red_led = Pin(red_led_pin, Pin.OUT)
button = Pin(button_pin, Pin.IN, Pin.PULL_UP)

led_state = False
while True:
    red_led.value(led_state)
    button_value = button.value()

    if button_value == 0:
        led_state = not led_state
```

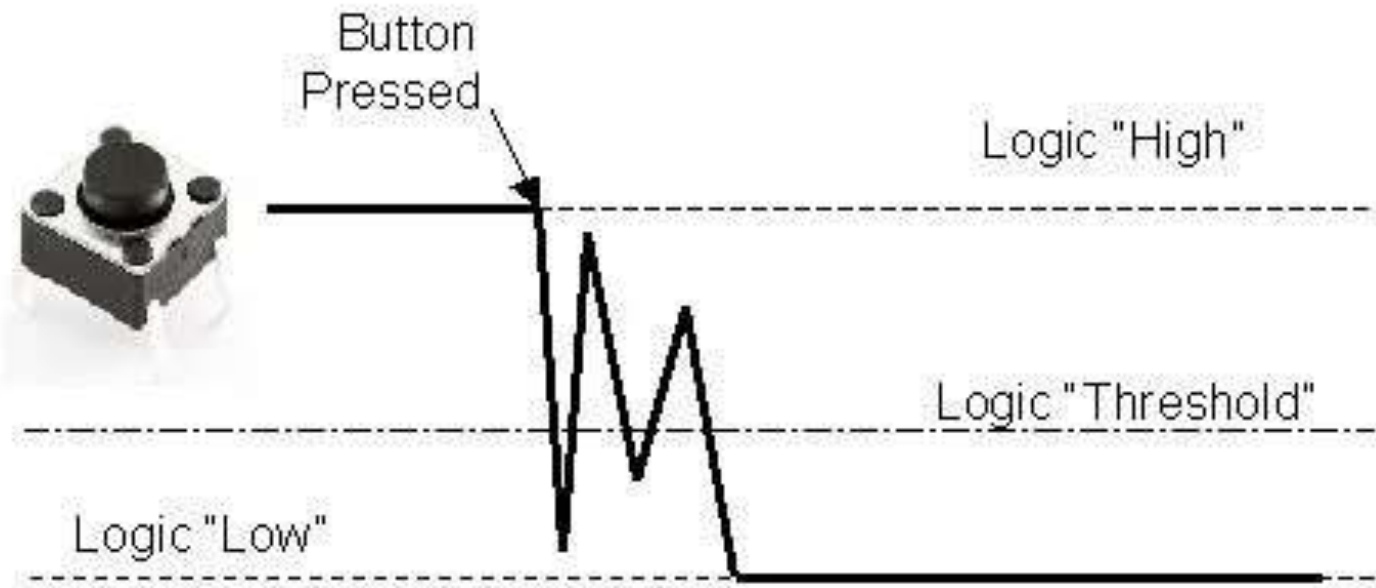
Дребезг контакта

Button "Bounce"



Программное устранениедребезга

Button "Bounce"



```
from machine import Pin
import time

red_led_pin = 15
button_pin = 16

red_led = Pin(red_led_pin, Pin.OUT)
button = Pin(button_pin, Pin.IN, Pin.PULL_UP)

led_state = False
while True:
    red_led.value(led_state)
    button_value = button.value()

    if button_value == 0:
        led_state = not led_state

    time.sleep(0.2)
```



Программное устранениедребезга

```
from machine import Pin
import time

red_led_pin = 15
button_pin = 16

red_led = Pin(red_led_pin, Pin.OUT)
button = Pin(button_pin, Pin.IN, Pin.PULL_UP)

led_state = False
while True:
    red_led.value(led_state)
    button_value = button.value()

    if button_value == 0:
        led_state = not led_state

    time.sleep(0.2)
```

```
from machine import Pin
import time

red_led_pin = 15
button_pin = 16

bounce_time = 250

red_led = Pin(red_led_pin, Pin.OUT)
button = Pin(button_pin, Pin.IN, Pin.PULL_UP)

led_state = False
timer_0 = time.ticks_ms()
while True:
    red_led.value(led_state)
    button_value = button.value()
    delta_time = time.ticks_ms() - timer_0
    if button_value == 0 and delta_time >= bounce_time:
        led_state = not led_state
        timer_0 = time.ticks_ms()
```



Контрольные вопросы

- Что такое микроконтроллер и в чем его отличие от микрокомпьютера?
- Что такое «схема распиновки» микроконтроллера?
- Что такое цифровой и аналоговый сигнал? В чем отличие между ними?
- Как правильно подключать светодиод?
- Что такое «наводка сигнала»?
- Как правильно подключать тактовую кнопку?
- Что такое «Подтяжка пина» и зачем она нужна?
- Что такое «дребезг» кнопки и как от него избавиться?
- Как работать с кнопкой не используя таймеры? Зачем это?